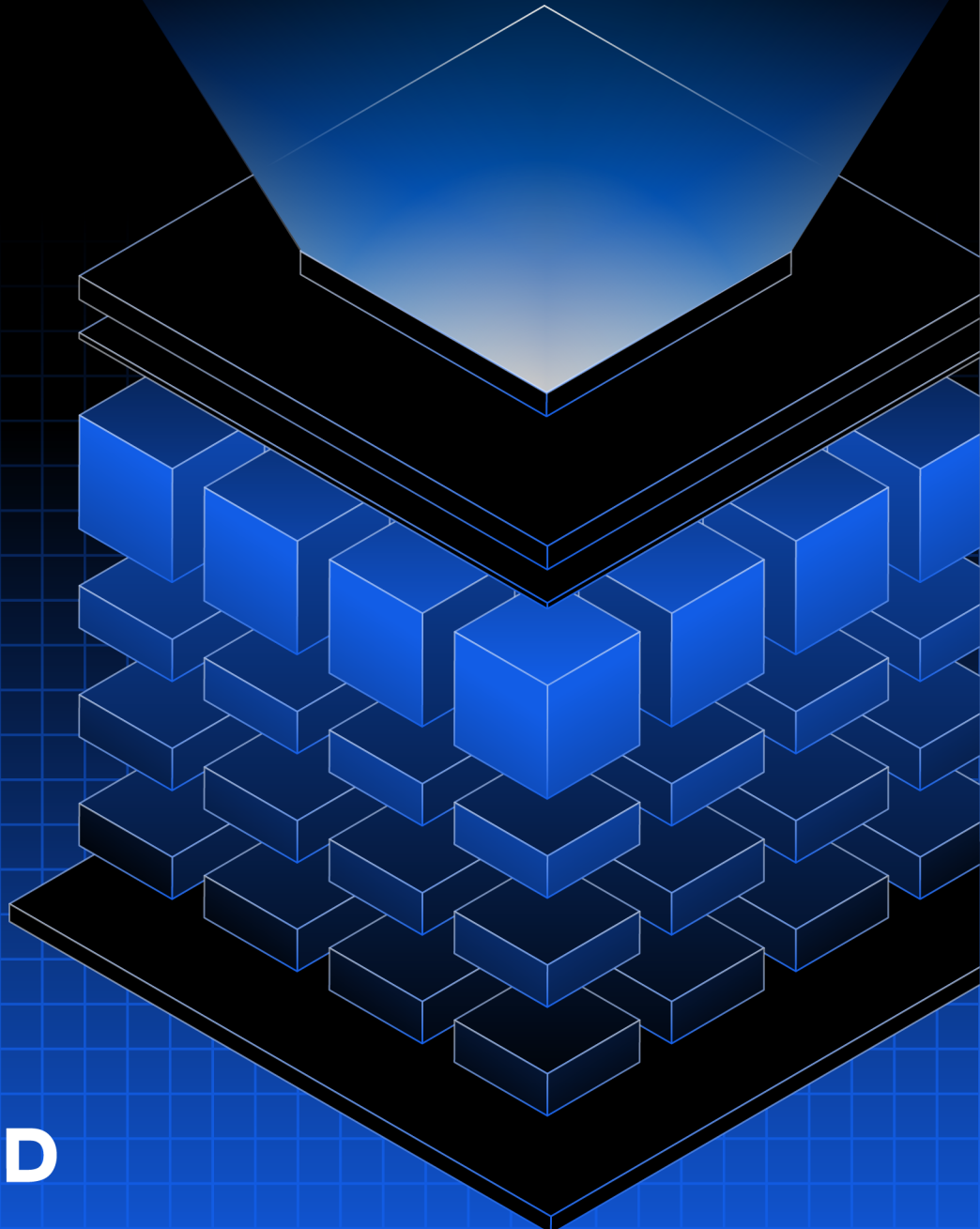


NHN FactoryX

기술 백서

NHN FactoryX White Paper



N H N CLOUD

NHN FactoryX 기술 백서

저작권

Copyright NHN Cloud Corp. All rights reserved.

이 문서는 NHN Cloud의 지적 자산이므로 NHN Cloud의 승인 없이 문서를 다른 용도로 임의 변경하여 사용할 수 없습니다. 이 문서는 정보 제공의 목적으로만 제공됩니다. NHN Cloud는 이 문서에 수록된 정보의 완전성과 정확성을 검증하기 위해 노력하였으나, 발생할 수 있는 내용상의 오류나 누락에 대해서는 책임지지 않습니다. 따라서 이 문서의 사용이나 사용 결과에 따른 책임은 전적으로 사용자에게 있으며, NHN Cloud는 이에 대해 명시적 혹은 묵시적으로 어떠한 보증도 하지 않습니다.

관련 URL 정보를 포함하여 이 문서에서 언급한 특정 소프트웨어 상품이나 제품은 해당 소유자의 저작권법을 따르며, 해당 저작권법을 준수하는 것은 사용자의 책임입니다.

NHN Cloud는 이 문서의 내용을 예고 없이 변경할 수 있습니다.

문서 이력

버전	일자	이력 사항
1.0 버전	2026. 6.	NHN FactoryX 기술 백서 1.0 버전 출시

목차

NHN FactoryX 기술 백서	2
--------------------------	---

저작권	2
-----------	---

문서 이력	2
-------------	---

01 개요	8
-------------	---

1.1 문서의 목적과 범위	8
----------------------	---

1.2 패러다임의 전환: 범용 클라우드에서 AI 전용 인프라로	9
--	---

1.2.1 대규모 클러스터 성능 저하의 구조적 원인	9
------------------------------------	---

1.2.2 AI 전용 인프라 요구 사항	10
-----------------------------	----

02 시스템 아키텍처	14
-------------------	----

2.1 GPU 클러스터 구성	14
-----------------------	----

2.1.1 클러스터 계층 구조	14
------------------------	----

2.1.2 Compute Fabric 아키텍처	14
---------------------------------	----

2.1.3 토폴로지: Fat-Tree 기반 Rail-Optimized 설계	15
---	----

2.2 고속 인터커넥트	16
--------------------	----

2.2.1 노드 내부: NVLink 및 NVSwitch.....	16
-------------------------------------	----

2.2.2 노드 간: InfiniBand	16
------------------------------	----

2.2.3 인터커넥트 설계의 균형 원칙	17
-----------------------------	----

2.3 전력 및 냉각 설계	18
2.3.1 공랭식 냉각 방식의 한계	18
2.3.2 D2C 수랭식 인프라 도입	21
2.3.3 수랭식 시스템 요소 설계	23
2.3.4 PUE 개선 및 인프라 신뢰성 확보	24
2.4 GPU 동적 할당 기술(GPU Live)	25
2.4.1 GPU 동적 할당 기술의 개념과 필요성	25
2.4.2 GPU Live의 동적 할당 기술	30
2.4.3 향후 개발 방향	35

03 하드웨어 구성 38

3.1 CPU 및 메모리 구성	38
3.2 GPU 사양 및 성능	39
3.3 스토리지 및 네트워크	40
3.3.1 스토리지 계층(Tiered Storage)	40
3.3.2 네트워크 구성.....	40

04 성능 분석 43

4.1 이론적 연산 성능 43

4.2 성능 저하 패턴 분석 44

4.2.1 네트워크 병목(Communication-bound)..... 44

4.2.2 메모리 부족(HBM/시스템 메모리) 44

4.2.3 스토리지 I/O 병목..... 44

4.2.4 열 스로틀링(Thermal Throttling) 45

4.2.5 체크포인트 오버헤드 45

4.3 패턴별 설계 단계 대응 체크리스트 46

05 확장성 및 운영 48

5.1 수평 확장(Scale-out) 전략 48

5.1.1 Scalable Unit 기반 증설 48

5.1.2 수평 확장과 수직 확장의 분리 48

5.2 자원 활용 효율 49

06 활용 사례 51

6.1 NHN FactoryX: 엑사스케일 클러스터 51

6.2 기업 프라이빗 AI 인프라 구축 사례 53

07 결론.....	55
08 부록	58
8.1 GPU 모델별 사양	58
8.2 용어 정의	59
8.3 참고 문헌	63

01 개요

- 1.1 문서의 목적과 범위
- 1.2 패러다임의 전환: 범용 클라우드에서 AI 전용 인프라로

01 개요

1.1 문서의 목적과 범위

생성형 AI와 대규모 언어 모델(Large Language Model, 이하 'LLM') 등장 이후 엔터프라이즈 환경에서 요구되는 연산 규모는 기존 클라우드 인프라가 가정해 온 범위를 크게 초과하고 있습니다. 수천억 파라미터 모델의 학습과 서빙은 단일 노드의 GPU 성능이 아니라, 수십에서 수백 노드 규모의 GPU 클러스터를 하나의 일관된 컴퓨팅 패브릭(Compute Fabric)으로 운용할 수 있는 인프라 설계 역량에 좌우됩니다.

이러한 환경에서 AI 인프라 도입은 단순 장비 구매가 아닌, 도입 이전 단계의 기술 검증을 동반하는 의사결정 프로세스로 자리 잡았습니다. 엔터프라이즈 고객은 GPU 모델 사양뿐 아니라 인터커넥트 토폴로지, 스토리지 I/O, 네트워크 병목 대응 방안, 운영 효율, 선형 확장성에 이르기까지 풀스택(Full-Stack) 관점의 설계를 고민해야 합니다.

본 백서는 NHN FactoryX의 AI 인프라 설계 철학과 운영 역량을 체계적으로 설명하여 AI 인프라 도입 과정에서 참고할 수 있는 공인 기술 레퍼런스를 제공하는 것을 목표로 합니다. AI 인프라 도입을 검토하는 조직의 기술 의사결정자, 대규모 AI 시스템을 설계하거나 운영하는 아키텍트 및 엔지니어 등이 본 백서를 활용할 수 있습니다.

본 백서는 다음 네 개의 핵심 영역으로 구성되며, 각 영역은 풀스택 설계 관점에서 상호 연관성을 갖지만 독립적으로 참조할 수 있습니다.

- **시스템 아키텍처**: GPU 클러스터의 컴퓨팅 패브릭과 고속 인터커넥트 구조, 전력 및 냉각 설계, GPU 동적 할당 기술
- **하드웨어 구성**: CPU, 메모리, GPU, 스토리지, 네트워크 사양과 설계 근거
- **성능 분석**: 이론적 연산 성능, 성능 저하 패턴, 설계 단계 대응 체크리스트, 선형 확장성 원칙
- **확장성 및 운영**: 수평적 확장(Scale-out) 전략과 자원 활용 효율

위 네 개 영역에 더해, 본 백서는 대표적인 활용 사례와 결론, 그리고 세부 데이터를 정리한 부록을 포함합니다. 본 백서가 다루는 기술 범위는 NHN FactoryX를 구성하는 인프라, 플랫폼, 서비스 레이어 중 핵심 기반이 되는 인프라와 플랫폼 영역에 해당합니다.

1.2 패러다임의 전환: 범용 클라우드에서 AI 전용 인프라로

기존의 범용 데이터 센터 아키텍처는 CPU 중심 연산, 공랭식 냉각, 이더넷 기반 네트워크, 트랜잭션 지향 스토리지의 설계 원칙을 기반으로 발전했습니다. 이 아키텍처는 AI 워크로드의 등장 이전까지 지난 수십 년간 가상화와 범용 클라우드 컴퓨팅을 중심으로 비즈니스 로직 처리와 데이터 저장의 효율성을 극대화해 왔습니다. 그러나 최근 LLM을 필두로 한 생성형 AI의 폭발적인 성장은 기존 인프라 설계가 가진 물리적·구조적 한계를 명확히 드러내고 있습니다. AI 모델의 파라미터 수가 기하급수적으로 증가함에 따라 단순히 서버의 수를 늘리는 방식인 수평적 확장(Scale-out)은 연산 효율과 비용 측면에서 임계점에 도달하였고, 이에 따라 대규모 병렬 연산과 초고속 데이터 전송, 급증하는 전력 밀도를 감당할 수 있는 AI 전용 인프라(AI-Native Infrastructure)로의 패러다임 전환이 요구되고 있습니다.

표 1-1 범용 데이터 센터와 AI 전용 데이터 센터 비교

구분	범용 데이터 센터	AI 전용 데이터 센터
연산 아키텍처	범용 CPU 중심의 순차 처리	GPU/NPU 기반 대규모 병렬 연산
랙당 전력 밀도	5~15kW	40~120kW 이상
냉각 방식	공랭식	직접 액체 냉각 / 침지 냉각
노드 내 인터커넥트	PCIe(수십 GB/s)	NVLink(900GB/s 이상)
노드 간 네트워크 패브릭	이더넷(10~100Gbps)	InfiniBand/RoCE(400Gbps 이상)
스토리지 패턴	랜덤 소규모 트랜잭션	대용량 순차 스트리밍 I/O
전력 규모	수 MW~수십 MW	수백 MW 이상(전용 전원 인프라)

1.2.1 대규모 클러스터 성능 저하의 구조적 원인

대규모 AI 모델의 학습 및 추론 과정은 단일 컴퓨팅 노드의 성능을 넘어 수천 대의 가속기(GPU)가 유기적으로 결합된 클러스터 환경을 전제로 합니다. 그러나 기존 범용 인프라 아키텍처는 이러한 대규모 클러스터링 환경에서 다음과 같은 구조적 병목 현상을 야기하며 전체 시스템 효율을 저하시킵니다.

통신 및 동기화 오버헤드에 따른 스케일링 효율 저하

AI 학습 시 각 노드(Node, 서버 단위)는 계산된 그레이디언트(Gradient)를 공유하는 전역 동기화(All-Reduce) 과정을 거칩니다. 이 과정에서 클러스터 내의 모든 노드가 데이터를 상호 교환하는 동안 연산은 일시적으로 중단되는데, 노드 개수가 늘어날수록 통신 경로의 수와 데이터 교환의 복잡도는 기하급수적으로 증가하게 됩니다. 이때 표준 이더넷 기반의 계층적 네트워크 아키텍처는 노드 확장에 따른 통신 지연(Latency)을 효과적으로 억제하지 못하며, 결과적으로 개별 가속기 간의 동기화 효율을 급격히 저하시킵니다. 이러한 구조적 한계는 연산 자원을 추가로 투입하더라도 실제 학습 속도는 비례하여 증가하지 않는 ‘성능 포화 상태’를 초래하는 결정적인 원인이 됩니다.

데이터 공급 병목으로 인한 데이터 굶주림 현상

대규모 AI 클러스터에서 스토리지는 단순한 저장소를 넘어 전체 연산 파이프라인의 혈류와 같은 역할을 수행합니다. 최근 가속기의 연산 속도는 비약적으로 향상되었으나, 기존 스토리지 계층의 데이터 읽기 속도가 이를 뒷받침하지 못하면서 연산 장치가 데이터를 기다리며 유휴(idle) 상태에 머무르는 병목이 발생합니다. 이와 같은 ‘데이터 굶주림(Data Starvation)’ 현상은 고가의 컴퓨팅 자원 활용률을 심각하게 저하시키는 핵심 원인이 됩니다.

체크포인팅(Checkpointing) 오버헤드에 따른 쓰기 병목

대규모 학습은 수주에서 수개월간 지속되므로 하드웨어 장애에 대비해 모델의 상태를 주기적으로 저장하는 체크포인팅 작업이 필수적입니다. 그러나 조 단위 파라미터 규모의 모델에서 생성되는 수 테라바이트(TB)급 체크포인트 파일을 수천 개의 노드가 동시에 쓰기 시도할 경우, 기존 스토리지 컨트롤러의 대역폭 한계를 초과하게 됩니다. 이 과정에서 발생하는 쓰기 병목으로 인해 전체 학습 시간의 약 10~25%가 연산이 아닌 저장 대기에 소비되는 심각한 운영 비효율이 발생합니다.

전력 밀도 급증에 따른 물리적 냉각의 한계

공기 대류에 의존하는 기존 냉각 설계는 AI 서버 랙에서 발생하는 극심한 열 밀도를 감당하기 어렵습니다. 특정 노드의 온도가 임계값을 넘어서면 시스템은 하드웨어 보호를 위해 성능을 강제로 낮추는 열 스로틀링(Thermal Throttling)을 수행하며, 이는 클러스터 전체의 동기화 속도를 가장 느린 노드에 맞추는 테일 레이턴시(Tail Latency) 문제를 심화시켜 클러스터의 전반적인 성능 하향 평준화를 야기합니다.

1.2.2 AI 전용 인프라 요구 사항

앞서 언급한 구조적 한계를 극복하고 가속기의 연산 효율을 극대화하기 위해서는 전용 인프라 설계 단계부터 기존과는 차별화된 다섯 가지 층위의 기술적 요건이 충족되어야 합니다.

고밀도 전력 수용과 열 관리를 위한 물리 인프라 설계

AI 워크로드는 전례 없는 수준의 전력 밀도와 물리적 중량을 동반하므로 데이터 센터의 설계 및 시공 단계부터 고중량 서버 랙을 견딜 수 있는 하중 설계와 대규모 전력 공급 체계가 전제되어야 합니다. 특히 무중단 운영을 위한 고가용성 전력 인프라와 함께 물리적 공간 효율을 극대화하면서도 고성능 장비의 배치와 유지보수가 용이한 AI 최적화 데이터 센터 인프라 설계 및 운영 능력이 필수적으로 요구됩니다.

초고밀도 전력 공급과 차세대 액체 냉각 시스템

랙당 60kW 이상의 전력을 소모하는 초고밀도 가속기 서버를 안정적으로 가동하기 위해서는 기존의 공랭식 환경을 뛰어넘는 혁신적인 에너지 관리 기술이 필요합니다. 전력 공급의 안정성 확보는 물론, 칩셋에서 발생하는 열을 즉각적으로 제거할 수 있는 수랭식(Liquid Cooling) 기반의 정밀 냉각 솔루션이 도입되어야만 서멀 스로틀링 없는 안정적인 연산 성능을 보장할 수 있습니다.

워크로드에 최적화된 통합 클러스터 아키텍처와 연산 성능 확보

컴퓨팅, 네트워크, 스토리지가 하나의 유기체로 동작해야 합니다. 특히 LLM의 학습 및 추론은 수조 개의 파라미터를 기반으로 한 고정밀 부동 소수점 연산을 요구하므로, 단순 범용 연산 성능보다 AI 특화 연산(FP8, FP16 등)에서 압도적인 TFLOPS를 제공하는 전용 가속기 배치가 필수적입니다. 이를 지원하기 위해 초저지연 통신을 위한 인피니밴드(InfiniBand) 및 RoCE(RDMA over Converged Ethernet) 최적화 튜닝과 고성능 병렬 스토리지 구조가 통합 설계되어야 하며, 이를 통해 단위 시간당 처리 가능한 토큰의 양을 극대화할 수 있습니다.

가속기 자원 효율화를 위한 지능형 관리 계층

다수의 GPU 자원을 효율적으로 배분하고 모니터링하기 위해서는 하드웨어 수준을 넘어선 소프트웨어 정의 관리 역량이 요구됩니다. 클러스터 내 가속기들의 상태를 실시간으로 파악하고, 장애 발생 즉시 즉각적인 조치 및 자원 재할당을 수행할 수 있는 전용 GPU 관리 플랫폼은 인프라 가동률을 극대화하고 운영 복잡성을 해결하기 위한 핵심 요건입니다.

통합 AI 개발 플랫폼

기술적 요구 사항의 최종 단계는 인프라의 물리적 성능을 실제 모델 개발 성과로 전환하는 것입니다. 압도적인 연산 자원을 갖추더라도 개발자가 분산 환경 설정, 라이브러리 의존성 등의 인프라의 물리적 복잡성에 갇혀 있다면 인프라 가용성은 현저히 떨어질 수 있습니다. 특히 대규모 클러스터링 환경에서는 데이터 전처리부터 모델 학습, 배포에 이르는 전 과정(End-to-End)이 인프라와 유기적으로 맞물려야 합니다. 이를 위해 복잡한 가상화 설정 및 분산 학습 환경 구축을 자동화하고, 가속기 성능을 최대로 끌어낼 수 있는 최적화된 프레임워크를 즉시 제공하는 통합 AI 개발 플랫폼이 필수적입니다. 인프라와 밀접하게 결합된 이러한 도구는 하드웨어 자원의 유휴 시간을 최소화하고, 개발자가 모델 알고리즘에만 집중할 수 있는 환경을 조성하는 데 기여할 수 있습니다.

AI 워크로드를 지탱하는 인프라는 단순히 개별 하드웨어의 성능 향상을 넘어, 물리적 공간 설계부터 냉각 효율, 초고속 네트워크 튜닝, 그리고 소프트웨어 정의 기반의 관리 플랫폼까지 모든 요소가 유기적으로 통합된 AI 네이티브 환경을 전제로 합니다. 기존 범용 인프라가 가졌던 구조적 한계를 극복하고 가속기의 잠재력을 100% 실현하기 위해서는 앞서 살펴본 다섯 가지 기술적 요구 사항에 대한 명확한 해답이 필요합니다.

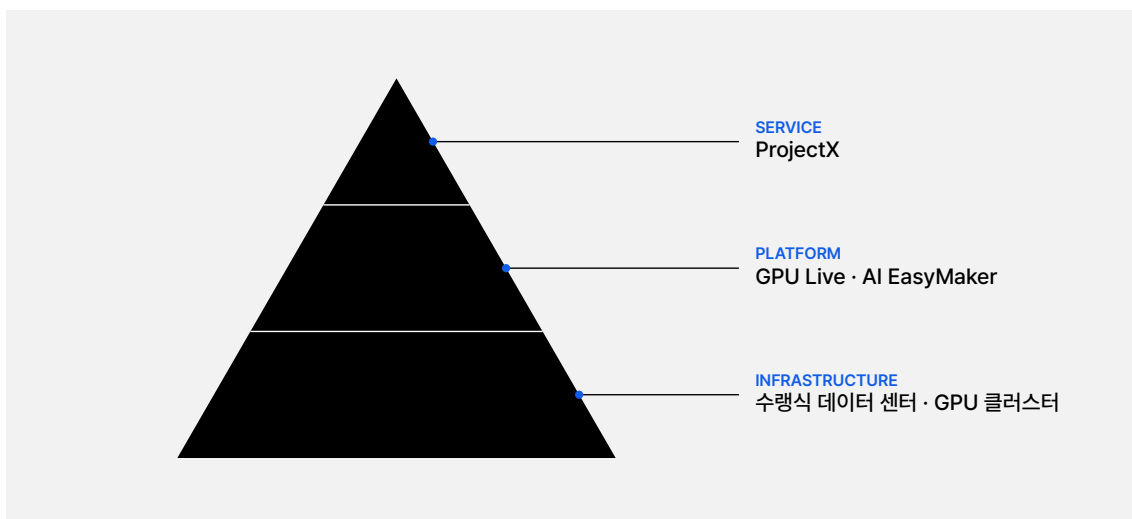


그림 1-1 AI 워크로드를 위한 NHN FactoryX 풀스택 3 계층

이러한 요구 사항을 완벽히 충족하기 위해 설계된 NHN FactoryX는 그림 1-1과 같이 AI 워크로드를 위한 인프라부터 플랫폼, 서비스까지 3개의 레이어를 하나의 공간에서 통합 제공합니다.

02 시스템 아키텍처

- 2.1 GPU 클러스터 구성
- 2.2 고속 인터커넥트
- 2.3 전력 및 냉각 설계
- 2.4 GPU 동적 할당 기술(GPU Live)

02 시스템 아키텍처

2.1 GPU 클러스터 구성

NHN FactoryX의 GPU 클러스터는 표준화된 GPU 노드(Building Block)를 기반으로, 동일한 토폴로지가 일관되게 반복 확장되는 스케일러블 단위 설계 원칙을 채택합니다. 이는 노드 추가 시점에 성능 예측 가능성을 확보하고, 운영 자동화 범위를 일관되게 유지하기 위한 선택입니다.

2.1.1 클러스터 계층 구조

클러스터는 다음과 같은 4개 계층으로 구성됩니다. 각 계층은 통신 거리·장애 격리 범위·관리 단위 측면에서 서로 다른 책임을 가지며, 상위 계층으로 갈수록 트래픽이 거치는 스위치 hop 수가 늘어나고 격리 수준은 강해집니다. 예를 들어 동일 Scalable Unit 내의 동일 Rail GPU 간 통신은 Leaf 스위치 1hop으로 완결되는 반면, 서로 다른 Rail이나 서로 다른 Scalable Unit 사이의 통신은 스파인(Spine) 계층을 추가로 경유해야 합니다.

표 2-1 GPU 클러스터 4 계층

계층	구성 요소	역할
노드(Node)	GPU 서버 1대	단일 쉘 내 GPU 간 NVLink로 결합된 최소 연산 단위
Scalable Unit	수~수십 노드	동일 InfiniBand 스위치 계층 아래의 비차단(Non-blocking) 도메인
Pod / Block	다수 Scalable Unit	스파인 스위치로 연결되는 운영·격리·다중 테넌트 단위
클러스터(Cluster)	Pod 집합	공통 관리 평면(Control Plane)과 스토리지 평면을 공유하는 최상위 단위

이 계층 구조는 단순한 물리적 구획이 아니라 워크로드 배치와 장애 영향 범위를 결정하는 논리적 경계 역할을 합니다. 따라서 분산 학습 작업은 가능한 한 하위 계층 내에서 통신이 완결되도록 작업을 배치하고 GPU를 할당하는 것이 성능 최적화의 기본 원칙이며, NCCL(NVIDIA Collective Communications Library)을 비롯한 집합 통신 라이브러리도 이러한 토폴로지 정보를 활용해 통신 경로를 최적화하도록 설계되어 있습니다. 또한 Pod 단위로는 멀티테넌트 격리가, Cluster 단위로는 공통 관리 평면 및 스토리지 평면 공유가 이루어져 노드 추가나 장애 복구를 일관된 자동화 체계 아래 수행합니다.

2.1.2 Compute Fabric 아키텍처

Compute Fabric은 GPU·CPU·메모리·스토리지·네트워크를 단일한 분산 컴퓨팅 자원으로 추상화하는 설계 개념입니다. NHN FactoryX는 다음 세 가지 패브릭을 물리적으로 분리하여 운영합니다. 분리 운영은 트래픽 간섭을 차단하고, 각 패브릭을 워크로드 특성에 맞게 독립적으로 튜닝하기 위한 표준 설계입니다.

- **Compute Fabric(East-West, GPU↔GPU):** 분산 학습 시 발생하는 All-Reduce, All-Gather 등 집합 통신을 담당합니다. InfiniBand(인피니밴드) 또는 RoCEv2(RDMA over Converged Ethernet v2) 기반의 Rail-Optimized 토폴로지를 사용합니다.
- **Storage Fabric:** 학습 데이터셋·체크포인트 I/O를 담당합니다. GPUDirect Storage(GDS) 지원 병렬 파일 시스템과 연결되어 CPU 우회 경로를 제공합니다.
- **Management/In-band Fabric:** 오케스트레이션·모니터링·로그 수집을 담당하는 일반 이더넷 평면으로, 컴퓨팅 트래픽과 물리적으로 분리됩니다.

패브릭을 물리적으로 분리하면 학습 시 발생하는 대규모 집합 통신이 스토리지 I/O나 관리 트래픽에 의해 간섭받지 않으며, 각 패브릭은 고유한 트래픽 패턴에 맞춰 독립적으로 튜닝할 수 있습니다. 예를 들어 Compute Fabric은 무손실 전송과 초저지연이, Storage Fabric은 높은 처리량과 안정적인 큐 관리가, Management/In-band Fabric은 폭넓은 호환성과 운영 가시성이 우선됩니다. 또한 패브릭 단위로 장애 영향이 격리되어, 한 평면의 문제가 전체 클러스터 가용성을 동시에 떨어뜨리지 않습니다.

2.1.3 토폴로지: Fat-Tree 기반 Rail-Optimized 설계

AI 클러스터의 인터커넥트는 일반적인 데이터 센터의 트래픽 패턴과 다르게, 동일 시점에 모든 GPU가 일제히 통신을 시도하는 집합 통신(Collective Communication)이 지배적입니다. 따라서 토폴로지는 Fat-Tree 기반의 비차단(Non-blocking) 또는 1:1 Subscribed 구성을 원칙으로 합니다.

Fat-Tree는 트리 구조에서 위쪽 계층으로 갈수록 링크 대역폭을 두껍게 확보하여 어떤 두 노드 사이에서도 동일한 유효 대역폭을 보장하도록 설계된 토폴로지입니다. 모든 Leaf 스위치가 동일한 수의 업링크로 스파인 스위치에 연결되므로 트래픽 패턴과 무관하게 경로 선택의 자유도가 유지되며, 일부 링크에 부하가 집중되는 현상을 구조적으로 완화합니다.

Rail-Optimized 토폴로지에서는 노드 내 N번째 GPU가 항상 동일한 N번째 Leaf 스위치 그룹에 연결되도록 NIC 배선을 정렬합니다. 이는 다수의 노드가 동일 인덱스 GPU 간 통신을 수행할 때(예: Tensor Parallel All-Reduce) 트래픽이 단일 스위치 계층 내에서 완결되도록 하여, 스파인 통과 지연(hop latency)과 혼잡을 동시에 줄입니다. 결과적으로 동일한 하드웨어 자원을 사용하면서도 집합 통신의 종단 지연이 짧아지고, GPU 활용률 저하의 주요 원인인 통신 병목을 완화할 수 있습니다.

2.2 고속 인터커넥트

AI 워크로드에서 인터커넥트는 단순 네트워크가 아니라 연산 성능의 일부입니다. 노드 내부의 GPU 간 통신과 노드 간 통신은 서로 다른 기술 스택을 사용하며, 두 계층의 대역폭이 균형을 이루지 못하면 전체 GPU 활용률이 급격히 저하됩니다.

2.2.1 노드 내부: NVLink 및 NVSwitch

NVLink는 동일 노드 내 GPU 간 직접 통신을 위한 NVIDIA의 전용 인터커넥트로, 일반 서버 인터커넥트인 PCIe보다 한 자릿수 이상 높은 대역폭과 짧은 지연을 제공합니다. NVLink가 GPU와 GPU를 직접 잇는 링크라면, NVSwitch는 이 NVLink들을 한 데 모아 임의의 GPU 쌍 사이에 동일한 대역폭을 제공하는 노드 내부 스위치 칩입니다.

NHN FactoryX의 표준 GPU 노드는 NVSwitch 기반 Full-Mesh 토폴로지를 채택하여, 노드 내 모든 GPU 쌍이 동일한 양방향 대역폭을 보장받도록 구성됩니다. Full-Mesh 구성에서는 어떤 GPU 두 장 사이의 통신이라도 NVSwitch를 단 한 번만 경유하므로, Tensor Parallel처럼 GPU 간 통신이 빈번한 학습 전략에서도 통신 거리에 따른 성능 편차가 발생하지 않습니다. 최신 세대인 NVLink 5/NVSwitch 4에서는 동일 원리가 단일 노드를 넘어 NVL72 랙 도메인 전체로 확장되어, 72개 GPU가 하나의 노드처럼 동작하는 통신 영역을 형성합니다.

표 2-2 NVLink/NVSwitch 세대별 GPU 인터커넥트

세대	GPU당 총 대역폭	대응 GPU(예)	적용 범위
NVLink 3/NVSwitch 2	양방향 600GB/s	A100	노드 내 GPU 8장 Full-Mesh
NVLink 4/NVSwitch 3	양방향 900GB/s	H100/H200	노드 내 GPU 8장 Full-Mesh
NVLink 5/NVSwitch 4	양방향 1,800GB/s	B200/GB200	노드 및 NVL72 랙 도메인

2.2.2 노드 간: InfiniBand

노드 간 통신은 InfiniBand를 표준으로 채택합니다. AI 클러스터에서 InfiniBand가 선택되는 이유는 단순한 대역폭이 아니라, 다음 세 가지 특성 때문입니다.

- **RDMA(Remote Direct Memory Access)**: CPU 개입 없이 원격 메모리에 직접 접근하여 마이크로초 단위 지연 시간을 실현합니다. 분산 학습의 통신은 곧 GPU 메모리 간 데이터 이동이므로, CPU 우회 경로의 확보는 처리량뿐 아니라 GPU 활용률에 직접 영향을 줍니다.
- **Adaptive Routing**: 스위치가 실시간 혼잡 정보를 기반으로 경로를 동적 재배치하여, All-Reduce 트래픽의 핫스팟을 분산합니다. 동일 시점에 모든 GPU가 일제히 통신하는 집합 통신 패턴에서는 정적 라우팅보다 혼잡 완화 효과가 크게 나타납니다.
- **SHARP(in-network computing)**: All-Reduce 같은 집합 연산의 일부를 스위치 내부에서 직접 수행하여, 네트워크 왕복 횟수를 줄이고 종단 지연을 단축합니다.

표 2-3 InfiniBand 세대별 대역폭 및 권장 NIC 구성

세대	포트당 대역폭	주요 적용 GPU	권장 노드당 NIC
HDR	200Gbps	A100	노드당 4~8NIC
NDR	400Gbps	H100/H200	노드당 8NIC(Rail당 1)
XDR	800Gbps	B200/GB200	노드 및 도메인 구성에 따름

2.2.3 인터커넥트 설계의 균형 원칙

풀스택(Full-Stack) 관점에서 인터커넥트는 단일 대역폭 수치가 아니라 다음의 균형 비율로 설계되어야 합니다. 어느 한 구간에 대역폭이 다른 구간 대비 부족하면 분산 학습의 통신 비중이 늘어나면서 GPU 활용률이 급격히 떨어지므로 각 비율은 클러스터 전체의 실효 성능을 결정하는 핵심 설계 변수입니다.

- **노드 내 ↔ 노드 간 대역폭 비율:** 분산 학습 전략(Tensor Parallel, Pipeline Parallel, Data Parallel)에 따라 요구 비율이 달라지며, 일반적으로 Tensor Parallel은 노드 내 NVLink 대역폭에, Pipeline/Data Parallel은 노드 간 InfiniBand 대역폭에 더 크게 의존합니다. 따라서 어떤 병렬화 전략이 주류가 될지에 따라 두 계층의 비율 설계 목표가 달라집니다.
- **GPU당 NIC 대역폭:** 단일 GPU의 메모리 대역폭과 인터커넥트 대역폭 사이의 비율이 일정 수준 이하가 되면 All-Reduce가 통신 한계(Communication-bound)가 됩니다.
- **Oversubscription Ratio:** Leaf-Spine 구간의 1:1 비차단 구성이 원칙이며, 비용 효율을 위해 도입되는 경우에도 최대 1:2를 넘지 않도록 합니다. Oversubscription이 커질수록 동시 다발적 집합 통신 상황에서 스파인 구간 혼잡이 빠르게 누적되어, 학습 작업 간 간섭이 측정 가능한 수준으로 나타날 수 있습니다.

2.3 전력 및 냉각 설계

AI 가속기의 랙당 전력 밀도가 60kW를 넘어서면서 전통적인 공랭식 냉각 인프라는 열역학적·물리적 한계, 운영의 구조적 제약, 에너지 비효율성이라는 측면에서 동시에 한계에 직면했습니다. 본 절에서는 공랭식 냉각 방식이 처리 가능한 전력 밀도의 구조적 한계의 원인을 짚어보고, 이를 대체할 직접 칩 냉각(Direct-to-Chip. 이하 'D2C') 메커니즘과 수랭식 인프라 도입 시의 검증 요건을 살펴봅니다. 이어서 CDU, 매니폴드(Manifold), 배관 수준의 실제 설계 파라미터와 외기 냉방 연계 운전 방식을 다루고, PUE 1.15 이하 달성 및 인프라 신뢰성 확보 방안까지 제시함으로써 AI 전용 데이터 센터의 전력·냉각 설계 전반에 대한 기술적 근거를 정리합니다.

2.3.1 공랭식 냉각 방식의 한계

AI GPU 전력 밀도의 급증

AI 학습 및 추론 워크로드의 폭발적 확산으로 데이터 센터의 랙당 전력 밀도는 과거 5년 대비 3~5배 수준으로 급증했습니다. 특히 NVIDIA Blackwell 아키텍처(B200/B300)의 상용화로 단일 서버 최대 소비 전력이 15kW를 초과하며 6대 구성 랙 기준 90kW 이상이 현실화되었습니다.

표 2-4 GPU 세대별 최대 소비 전력 및 랙 밀도 비교

세대	GPU 서버 최대 소비 전력 (8-GPU Baseboard 기준)	랙 밀도 (GPU 서버 2~6대 구성 시)	비고
A100(2020)	6.5kW	13~39kW	고성능 공랭식 대응 가능
H100(2022)	10.2kW	20~61kW	공랭식 한계 진입
H200(2024)	11.0kW	22~66kW	냉각 방식 과도기
B200(2025~)	14.3kW	29~86kW	D2C 전용 설계 도입 검토
B300(2025~)	약 15kW	30~90kW	D2C 전용 설계 필수

이 추세는 단기적 현상이 아닙니다. 무어의 법칙(Moore's Law) 둔화에 따라 반도체 업계는 병렬 연산 집적도 향상으로 성능을 확보하고 있으며, 이는 열 발생 밀도의 구조적 증가를 의미합니다. B200 이후 세대부터는 공랭식/수랭식의 선택 문제가 아니라 D2C 수랭식 인프라를 전제로 한 서버 설계가 표준화되는 방향으로 수렴하고 있습니다. 데이터 센터 인프라는 현세대 요건이 아닌 B300 세대까지의 상위 호환성(Forward-Compatible) 설계를 전제로 수립되어야 합니다.

공랭식 냉각 방식의 열역학적 한계

공기의 체적 열용량($\rho \cdot C_p$)은 $1.2\text{kJ}/(\text{m}^3 \cdot ^\circ\text{C})$ 으로, 물($4,186\text{kJ}/(\text{m}^3 \cdot ^\circ\text{C})$) 대비 약 3,500배 낮습니다. 동일 열량 제거 시 공기는 물 대비 약 3,500배의 유량이 필요하며, 이는 냉각 매체의 반송 동력이 기하급수적으로 증가하는 원인이 됩니다. 서버 전력이 10kW를 초과하는 시점부터 팬 소모 전력이 IT 부하의 15% 이상을 차지하기 시작하며, 서버의 냉각 팬이 서버의 소비전력을 가중시키는 역설적 구조가 발생합니다.

표 2-5 냉각 방식별 물리적 특성 비교

냉각 방식	냉각 매체	비열 [kJ/(kg·°C)]	체적 열용량 [kJ/(m ³ ·°C)]	열전달계수 [W/(m ² ·°C)]
공랭식	공기	1.006	1.2	25~250
수랭식	물	4.182	4,186	2,000~15,000
	물(PG 25%) 혼합액 ¹	3.9	3,900	1,700~11,000

냉각 시스템별 적용 한계

냉각 방식은 처리 가능한 랙 전력 밀도에 따라 적용 한계가 명확합니다. 공랭식 계열은 구조적으로 30kW/rack 내외가 실질적 상한이며, 간접 수랭식(Indirect Liquid Cooling) 계열도 50kW의 벽을 넘지 못합니다. 75kW/rack 이상 구간에서는 직접 수랭식(Direct Liquid Cooling) 계열만이 유효한 해법으로 남습니다.

표 2-6 냉각 시스템별 적용 한계

냉각 시스템	냉각 방식	적용 한계	비고
CRAC, CRAH	공랭식	~20kW/rack	공기 열용량 한계, 원거리 냉기 전달 손실
Containment	공랭식	~30kW/rack	기류 제어 개선에 불과, 매체 한계 동일
In-row Cooler	공랭식	~30kW/rack	서버 팬 의존, 고밀도 시 재순환 열 간섭
RDHX(Rear Door HX)	수랭식(간접)	~50kW/rack	배기열 흡수 방식, 칩 직접 냉각 불가
Direct-to-Chip(D2C)	수랭식(직접)	30kW~150+ kW	칩 직접 냉각, 열 전달 최적화
Immersion Cooling	액침 냉각	30kW~150+ kW	서버 전체 액침, 특수 설비 필요

ASHRAE 등급 기준 및 개정 현황

ASHRAE TC 9.9 5th Edition(2021)은 공랭식 등급을 A1~A4로 분류하며, 고밀도 장비 전용 H1 Class를 신설했습니다. H1은 GPU 가속기 등 고집적 연산 장비를 위한 별도 등급으로, 권장 흡입구 온도 상한이 A 계열(27°C)보다 엄격한 25°C로 설정되어 있습니다. 이는 고밀도 장비일수록 허용 온도 범위가 오히려 좁아짐을 의미합니다. 공랭식으로 이 기준을 충족하는 것은 엄격하고, 정밀한 운영 관리를 요구합니다.

2024년 ASHRAE는 기존 Datacom Series를 Datacom Encyclopedia로 전면 개편하였으며, 수랭식 가이드라인에 S Class(Technology Cooling System 공급 온도 기준)를 신규 도입하여 수랭식 시스템의 설계 프레임워크를 대폭 확장했습니다. 이는 업계 표준이 공랭식 최적화에서 수랭식 기반 설계로 무게 중심을 이동하였음을 공식화한 것입니다.

¹ 물(PG 25%) 혼합액은 차량이나 냉각 설비의 동파 방지를 위한 부동액으로 사용되나 D2C 시스템에서는 수랭 쿨드 플레이트 내 유로의 오염 물질 흡착, 쿨드 플레이트 및 배관 부식 방지를 위한 수질 관리용으로 사용됨.

표 2-7 ASHRAE 등급

ASHRAE 등급	흡입구 온도 범위	적용 전력 밀도	냉각 방식
A1	15~32°C	≤ 5kW/rack	공랭식 표준
A2	10~35°C	≤ 10kW/rack	공랭식 표준
A3	5~40°C	10~20kW/rack	공랭식+보조 냉각
A4	5~45°C	20~25kW/rack	공랭식 한계
H1(신설)	권장 상한 25°C	고밀도 공랭식 장비 전용	수랭식 적용 검토
W17~W+	수랭식 공급 온도 기준	25kW/rack 이상	수랭식 적용 필수

Almoli et al.(2012) CFD 분석 연구에 따르면 랙 전력 밀도가 20kW를 초과하면 서버 흡입구 온도 불균일도(ΔT)가 8°C 이상 발생하며, 냉복도 컨테이너먼트 적용에도 불구하고 ASHRAE A2 권장 범위 초과 빈도가 급증하는 등 공랭식 냉각의 구조적 한계가 명확히 지적된 바 있습니다. 이러한 연구 결과를 고려할 때, 랙당 전력 밀도가 60~90 kW에 육박하는 최신 B200/B300 기반 클러스터 환경에 기존 공랭식 방식을 고집하는 것은 AI 서비스의 신뢰성 저하 및 조기 장애로 직결될 수밖에 없습니다.

공랭식 냉각의 한계와 D2C 수랭식 도입의 필연성

고밀도 환경(30kW/rack 이상)에서 공랭식 냉각 방식을 적용할 경우, 냉각 열량 처리를 위한 공조 장비의 설치 대수 및 점유 면적(Footprint)이 급격히 증가합니다. 공랭식 냉각 방식의 열역학적 특성상 발생하는 대풍량 공급 구조는 서버룸 내 기류 유동 제어의 한계를 초래하며, 풍속 과다에 따른 미세 먼지 유입 및 응축수 비산 이슈를 유발하여 데이터 센터 가동 건전성에 심각한 구조적 제약 요인으로 작용합니다. 아울러, 부족한 냉각 능력을 보완하기 위해 서버 내장 팬의 회전수(RPM)가 최고치에 도달함에 따라, 실내 소음이 고위험 수준(80~95 dBA)으로 증폭되고 극심한 장비 진동 문제를 수반하여 물리적 안정성을 저해합니다.

또한, ASHRAE Climatic Design Conditions 2025 가이드라인에 따른 대한민국 서울 지역의 50년 빈도 극한 기후 데이터(하절기 극한 건구온도 39.0°C, 동절기 극한 건구온도 -21.1°C)를 적용할 때, 혹서기 공랭식 냉각 시스템을 안정적으로 가동하기 위해서는 천문학적인 기계식 냉방(Chiller Peak Load) 에너지가 요구됩니다. 반면, 수랭식 냉각 방식은 냉동기 및 냉각탑에서 생산된 냉수를 칩 표면의 쿨드 플레이트에 직접 전달하는 고효율 방열 구조를 채택하고 있습니다. 이는 서울 지역의 연간 기온 분포 특성과 맞물려 외기 연동형 프리쿨링(Free Cooling) 시간 가용률을 90% 이상으로 극대화하며, 전력 소모를 최소화하는 동시에 서버룸 내 공조 인프라 점유 면적을 혁신적으로 절감시킵니다.

표 2-8 냉각 방식별 PUE 비교²

냉각 방식	연간 PUE(서울 지역 기준 예시)	비고
공랭식	1.60~1.80	연중 기계식 냉방 운전
공랭식(프리쿨링)	1.35~1.50	프리쿨링 가용률 약 70% 내외 한계
D2C 수랭식(프리쿨링)	1.10~1.30	프리쿨링 가용률 약 90% 이상 확보

위에서 살펴본 바와 같이 공랭식은 열역학적, 물리적 한계, 운영의 구조적 제약, 에너지 비효율성이라는 축에서 동시에 한계에 도달했습니다. 특히 75kW 이상 구간에서는 간접 액체 냉각조차 대응 불가하며, D2C 수랭식 전환은 선택이 아닌 공학적 필연이 되었습니다.

2.3.2 D2C 수랭식 인프라 도입

D2C 냉각 메커니즘 및 열 교환 단계 비교

D2C 냉각은 CPU, GPU와 같은 발열원이 되는 칩 자체에 쿨드 플레이트를 밀착 장착하고, 내부 미세 유로를 통해 냉각수가 순환하며 전도 방식으로 직접 열을 제거하는 수랭식입니다. 공기를 매개하지 않으므로 열 저항 경로가 단축되고, 열 전달계수(h)가 공랭식 대비 수십 배 높아 NHN FactoryX와 같은 75kW/rack 이상의 고전력·고발열 처리가 가능합니다.

표 2-9 냉각 방식별 열 제거 경로 및 단계 수 비교

냉각 방식	열 제거 경로	열 교환 단계 수 ³	비고
공랭식(냉동기)	칩 발열부 → 방열판 → 공기 → CRAH → 냉수 → 냉동기 → 냉각수 → 냉각탑 → 외기	4단계	일반 데이터 센터
D2C 수랭식(냉동기)	칩 발열부 → 쿨드 플레이트 → 냉수(PG25%) → CDU → 냉수 → 냉동기 → 냉각수 → 냉각탑 → 외기	4단계	AI 전용 데이터 센터
D2C 수랭식(냉각탑)	칩 발열부 → 쿨드 플레이트 → 냉수(PG25%) → CDU → 냉각수 → 냉각탑 → 외기	3단계	AI 전용 데이터 센터

열 교환 단계 수 비교에서 핵심은 수치 자체보다 공기 매개 단계의 유무입니다. 공랭식은 열원에서 냉각 매체까지 반드시 공기를 경유해야 하며, 공기 구간(방열판 → 공기)은 열전달계수가 극히 낮아(25~250W/m²K) 전체 열 저항의 60~70%를 차지합니다. NHN FactoryX와 같은 D2C 수랭식은 이 공기 매개 단계를 완전히 제거하고 칩 발열부에서 수 mm 거리의 쿨드 플레이트가 직접 열을 회수합니다.

D2C 수랭식(냉각탑) 구성은 CDU 이후 냉동기를 배제하고 냉각탑에서 직접 방열하는 방식으로, ASHRAE W4 Class

2 냉각 방식별 프리쿨링 가용률 및 연간 PUE 값은 기상청의 기후 평년값(1991~2020년) 기반 대한민국 서울 지역 시간별 외기 온도 빈도(BIN) 데이터(연간 8,760시간) 및 ASHRAE 2025 극한 기후 조건을 준용하여 산출한 것으로, 외기 건구온도 15°C 이하에서 가동하는 공랭식 프리쿨링은 연간 약 6,132시간(가용률 약 70%)의 운전 한계를 가지는 반면, ASHRAE W3/W4 등급 고온수 가동 설계가 적용되어 외기 건구온도 25°C 이하에서 가동하는 D2C 수랭식 프리쿨링은 연간 약 7,884시간(가용률 약 90%)까지 외기 냉방 구간을 극대화할 수 있음.

3 열 교환 단계 수는 시스템 내 독립적인 유체 루프 및 주요 기계 설비를 거쳐가는 단계 수를 기준으로 산정함.

기준(공급 온도 $\leq 45^{\circ}\text{C}$) 설계 시 적용 가능합니다. 공랭식(냉동기), 수랭식(냉동기) 4단계 대비 열 교환 단계가 1단계 감소되며, 에너지 소비가 많은 냉동기 압축기를 사용하지 않으므로 PUE 개선 효과가 가장 큼니다. 단, 냉각탑 단독 운전은 외기 습구 온도에 의존하므로 국내 하절기(습구 온도 최대 30°C 이상) 조건에서 냉수 온도 공급 가능 여부를 설계 단계에서 반드시 검토해야 합니다.

수랭식 인프라 도입 시 검증 요건

수랭식 인프라는 공랭식 대비 배관, 유체, 계면 등 추가 엔지니어링 요소가 개입되므로, 도입 전 다음 항목의 체계적 검증이 선행되어야 합니다.

• 벤더 호환성(Vendor Compatibility)

GPU·CPU 제조사(NVIDIA, AMD, Intel 등)는 자사 플랫폼에 대해 쿨드 플레이트 장착 사양(접촉 압력, 치수 공차, 열 전달 패드 규격)을 별도로 정의합니다. 서버 제조사(Dell, HPE, Supermicro 등)의 OEM 수랭식 지원 여부 및 보증 조건을 사전 확인해야 하며, 쿨드 플레이트가 칩 패키지 사양을 벗어난 압력이나 치수로 장착될 경우 칩 손상 및 보증 무효화로 이어질 수 있어 주의해야 합니다. NHN FactoryX는 수랭식 품질 관리를 위해 계획 단계에서부터 CPU, GPU, 서버 제조사뿐만 아니라 냉각 솔루션 공급사들과 사전 검증을 수행하여 호환성을 철저히 검토했습니다.

• 인터페이스 표준화(Interface Standardization)

수랭식 인프라 초기 시장에서는 국제 표준이 부재하여 CDU(cooling distribution unit) 제조사(CoolIT, Vertiv, Schneider 등), 서버 제조사, 냉각 부품 제조사가 각자의 자체 규격으로 커넥터·배관 인터페이스를 설계했습니다. 이로 인해 제조사 간 호환이 불가하고, 서버 교체 시 배관 재공사가 불가피한 Lock-in 문제가 반복되었습니다. 현재는 OCP(open compute project)·ASHRAE TC 9.9·ISO 등을 통해 QD(quick disconnect) 커넥터 규격, 배관 치수, 운전 파라미터 기준의 표준화가 진행 중이나, 완전한 상호운용성(Interoperability) 확보를 위해서는 도입 전 커넥터 규격의 호환 여부를 설계 단계에서 명시적으로 확인해야 합니다. NHN FactoryX는 수랭식 시스템 구축 전 설계 도서에서 수랭식 시스템의 QD 커넥터 규격, 배관 치수, 운전 파라미터 기준 등을 검토하여 품질을 확보했습니다.

• 운전 파라미터 검증(Operational Parameter Validation)

수랭식 시스템은 냉수 공급 온도, 냉수 환수 온도, 냉수 순환 유량, 배관 내 운전 압력 등의 운영 변수가 동시에 설계 범위 내에서 운영되어야 합니다. 이 중 하나라도 설계 범위를 이탈할 경우 칩 과열, 냉수 누수, 배관 피로 파괴 등의 장애로 이어질 수 있습니다. NHN FactoryX는 수랭식 시스템 구축 이후, 성능 검증을 위해 실제 IT 부하 조건에서 냉각 시스템 신뢰성 검증 테스트(Commissioning)를 수행했으며, 공급사에서 제공한 설계값과 실측값의 편차를 분석하여 수랭식 시스템 운영에 반영했습니다.

• 확장성(Scalability)

수랭식 시스템은 초기 구축 이후 랙 증설, 서버 교체, 전력 밀도 증가를 고려한 배관 및 CDU 용량 재설계가 수반될 수 있습니다. GPU 세대 전환(H100 → B200 → B300) 시 랙당 전력이 최대 30~40% 증가하므로, CDU 용량, 냉수 배관 규격, 펌프 유량 등을 증설을 고려하여 선정해야 추후 대규모 재공사를 방지할 수 있습니다. NHN FactoryX는 B200, B300 이후 GPU 플랫폼 전환 시나리오를 반영하여 수랭식 시스템을 구축했습니다.

• 수질 관리(Coolant Quality Management)

공랭식과 달리 수랭식 시스템은 냉각수 수질이 열전달 성능과 설비 수명에 직접 영향을 미칩니다. 비저항, pH, 금속

이온 농도, 미생물 기준을 정기적으로 모니터링하지 않으면 배관 내 부식, 스케일, 생물막(Biofilm) 형성으로 유로가 막히거나 콜드 플레이트 열저항이 증가합니다. NHN FactoryX에서는 수랭식 시스템의 수질 관리를 위해 물 대신 물(PG 25%) 혼합액을 사용하였으며, 점도가 높은 물(PG 25%) 혼합액 특성을 고려하여 양정이 큰 펌프를 선정하였습니다.

2.3.3 수랭식 시스템 요소 설계

CDU 설계 기준

NHN FactoryX는 고성능 구현을 위한 낮은 GPU 온도 환경을 조성하기 위해 냉동기를 사용하여 냉수를 공급 받는 방식을 채택하였습니다. 수랭이 담당하는 부하는 콜드 플레이트가 설치되는 CPU, GPU로 전체 서버의 정격부하 대비 약 70%에 해당합니다. 메모리, 파워, 메인보드 등은 여전히 공랭식 냉각 방식이 요구되며 나머지 30%에 해당합니다. NHN FactoryX의 랙당 75kW 부하에서 70%를 수랭 부하, 나머지 30%를 공랭 부하로 산정하였습니다. CDU 설계 기준은 ASHRAE 및 OCP 기준을 참고하여 선정하였으며 표 2-10과 같이 정리할 수 있습니다.

표 2-10 CDU 주요 설계 기준

구분	기준
냉수(PG 25%) 공급 온도	ASHRAE W20~W+
랙당 수랭식 열량	IT 부하의 약 70% 담당(서버 제조사에 따라 다름)
허용 압력 강하(Rack 내)	OCP 권고 $\leq 2.0\text{bar}$
비저항	금속 부식 방지
pH	OCP 수질 기준

매니폴드 및 QD 선정 설계

매니폴드는 CDU에서 공급된 냉수를 랙 내 GPU 서버로 분배, 환수하는 설비이며, QD 커넥터는 누수 없이 매니폴드와 GPU 서버의 연결, 분리를 요구할 때 사용합니다. AI 서버는 GPU 세대 전환에 따른 교체 주기가 짧고 단일 서버 장애가 클러스터 전체 학습 작업에 영향을 줄 수 있으므로 공급/환수 배관을 병렬로 분리할 수 있도록 설계합니다. NHN FactoryX에서는 매니폴드를 사용하여 GPU 서버에 냉수를 고르게 배분하였으며, QD를 이용하여 누수 없이 신속하게 장비를 교체할 수 있도록 시스템을 구성하였습니다.

에너지 절감을 위한 프리쿨링 운전

NHN FactoryX의 냉수(PG 25%) 공급 온도 20°C는 외기 조건에 따라 세 가지 운전 방식을 사용합니다. 표 2-11에는 2025년 대한민국 서울 지역 기상청 데이터를 바탕으로 운전모드별 운전 기준 및 운전 시간을 나타냈습니다. 표 2-11에 따르면, 연간 외기온이 26°C 이하인 기간이 약 80%이며, 프리쿨링 운전으로 GPU 서버를 냉각하므로 PUE 1.15 달성이 가능합니다.

표 2-11 프리쿨링 운전 기준 및 운전 시간

운전 모드	외기 조건(°C)	운전 시간(h)	연중 해당 시간(%)	가동 설비
프리쿨링 운전	≤ 18°C	5,024 h	57.4%	냉동기 OFF, 프리쿨링 열교환기 ON
부분 프리쿨링 운전	18~26°C	2,150 h	24.5%	냉동기 ON, 프리쿨링 열교환기 ON
냉동기 운전	> 26°C	1,586 h	18.1%	냉동기 ON, 프리쿨링 열교환기 OFF

2.3.4 PUE 개선 및 인프라 신뢰성 확보

PUE 개선 효과

NHN FactoryX에서 구축한 D2C 수랭식 시스템은 GPU 칩에서 발생하는 열을 직접 방열하는 구조로, 냉각에 필요한 에너지 소비를 근본적으로 절감할 수 있습니다. 특히 ASHRAE W27 조건에서 운전하고 외기 냉방과 연계할 경우, PUE 1.15 이하 달성이 충분히 현실적인 수준으로 평가됩니다. 이는 공랭식(PUE 1.6~1.8) 대비 연간 냉각 에너지를 약 30~40% 절감할 수 있음을 의미합니다.

인프라 신뢰성 확보

D2C 액체 냉각 방식은 GPU 칩 온도를 설계 범위 내에서 정밀하게 제어함으로써 높은 GPU 온도로 인해 발생하는 서버 장애율을 낮추고 서버 가용성 확보 및 신뢰성을 향상시킵니다. 또한 Dry-break Quick Disconnect (QD), N+1 구성의 CDU 펌프, 누수 센서 연동과 같은 보호 메커니즘을 통해 수랭식 설비 리스크를 최소화하여 운영 안정성을 강화할 수 있습니다. 또한, 인터페이스 표준화, 확장성 등을 시스템 선정에 반영하여 차세대 GPU 플랫폼으로 전환하더라도 기존 냉각 인프라를 재사용할 수 있습니다. 또한, 동일 IT 용량 대비 작은 설치면적으로 공간 효율 측면에서도 유리합니다 (예: 공랭식 20kW/rack 대비 랙 수 약 75% 감소).

2.4 GPU 동적 할당 기술(GPU Live)

대규모 GPU 클러스터 운영의 핵심 과제는 한정된 고성능 연산 자원을 다수의 사용자 및 이질적인 워크로드에 최적으로 배분하는 것입니다. GPU 클러스터는 기존 CPU 중심의 클러스터와 비교하여 다음과 같은 고유한 특성을 가집니다.

- **높은 자원 단가와 유헴 비용:** GPU는 단일 부품 및 시스템 단가가 매우 높아 낮은 활용률(Utilization)은 막대한 운영 비용 손실로 직결됩니다.
- **워크로드의 이질성:** 학습(Training), 추론(Inference), 데이터 전처리 등 각 워크로드 처리 유형마다 요구되는 GPU 사용 형태가 각각 상이하여 단일 정책 적용이 어렵습니다.
- **스케일링의 복잡성:** 단일 GPU의 성능을 초과하는 대규모 분산 학습부터, 1장의 GPU를 쪼개 쓰는 미세 할당까지 폭넓은 유연성이 요구됩니다.

이러한 특성으로 인해 기존의 일반적인 클라우드 자원 관리 모델을 그대로 적용하기 어려우며, GPU에 특화된 할당 및 스케줄링 모델이 필수적입니다. 본 절에서는 GPU 동적 할당 기술의 개념을 정의하고, NHN FactoryX의 GPU Live가 Kubernetes 표준 환경 내에서 이를 어떻게 통합하여 운영 효율을 극대화하는지 상세히 기술합니다.

2.4.1 GPU 동적 할당 기술의 개념과 필요성

자원 할당 방식은 크게 정적 할당(Static Allocation)과 동적 할당(Dynamic Allocation)으로 구분합니다. 본 절에서는 두 방식의 차이를 짚고, 동적 할당 환경에서 자원의 분할(Sharing)과 결합(Scaling), 그리고 이를 뒷받침하는 스케줄링·정책 모델을 정리합니다.

정적 할당과 동적 할당

정적 할당은 팀 또는 프로젝트 단위로 GPU를 고정 배정하는 방식입니다. 운영이 단순하고 사용자 입장에서 자원 사용을 예측하기 쉬우나, 워크로드가 없는 시간대의 유헴 자원이 다른 사용자에게 재배분되지 못하는 구조적 한계가 있습니다. 학습 작업의 디버깅·휴지 시간, 추론 워크로드의 저트래픽 시간대, 부서 간 사용 패턴 차이 등으로 인해 클러스터 전체 관점에서 다수의 GPU가 유헴(idle) 상태로 머무르게 됩니다. 이는 작업이 GPU를 점유하는 동안의 효율성 지표(MFU 등)와는 별개의 차원으로, 대규모 GPU 클러스터 트레이스 분석에서 클러스터 자원 활용률(Allocation/Busy Utilization)이 워크로드 구성에 따라 한 자리 수에서 30%대 수준에 머무르는 사례가 보고되어 왔습니다.

동적 할당은 GPU를 공통 풀(Pool)로 관리하며 워크로드 발생 시 온디맨드로 자원을 배정하거나 회수하는 방식입니다. 동일한 물리 자원으로 더 많은 워크로드를 수용할 수 있어 자원 활용률이 높아지지만, 다음과 같은 운영 부담이 따릅니다.

- **스케줄링의 복잡도:** 다수의 워크로드가 동시에 자원을 요청하므로 우선순위, 자원 적합성, 토폴로지를 종합적으로 고려한 배치 결정이 필요합니다.
- **정책 일관성:** 사용자, 팀, 프로젝트별 사용량 한도와 보장량을 동시에 만족시켜야 합니다.
- **장애 회복성:** 자원이 동적으로 회수·재배정되는 상황에서 실행 중인 워크로드의 안정성을 유지해야 합니다.

GPU 분할(GPU Sharing): 단일 GPU의 효율 극대화

단일 워크로드의 요구량이 GPU 한 장의 성능보다 적을 경우 자원을 논리적으로 분할하여 공유합니다. 분할 방식에 따라 격리 수준과 활용 시나리오가 달라집니다.

• 시분할(Time-slicing)

시분할 방식은 NVIDIA Kubernetes Device Plugin에서 제공하는 기능으로, 하나의 물리적 GPU 자원을 복수의 컨테이너가 시간 단위로 번갈아 가며 점유하도록 지원합니다. 이 방식은 별도의 하드웨어 제약 없이 소프트웨어 설정만으로 구현이 가능하여 범용성이 높다는 장점이 있습니다. 하지만 컨테이너 간 GPU 메모리나 연산 유닛, 장애 영역이 논리적으로 격리되지 않는다는 구조적 한계가 있습니다. 즉, 특정 컨테이너의 자원 점유나 비정상 동작이 동일 GPU를 공유하는 다른 컨테이너의 성능과 안정성에 직접 영향을 줄 수 있습니다. 따라서 시분할 방식은 엄격한 성능 보장과 안정성이 요구되는 운영 워크로드보다는 개발, 실험, 교육 환경에서 클러스터 효율을 높이는 용도로 사용하는 것이 적합합니다.

• MPS(Multi-process service)

MPS는 NVIDIA CUDA 런타임이 제공하는 GPU 공유 메커니즘으로, 하나의 물리 GPU 위에 다수 프로세스의 CUDA 컨텍스트를 동시에 활성화해 공간적으로 공유합니다. 시분할이 컨테이너 단위로 GPU 사용 시간을 번갈아 점유하는 방식이라면, MPS는 SM(streaming multiprocessor)을 프로세스 사이에 동시에 분배하므로 컨텍스트 전환(Context Switch) 오버헤드가 줄고, 단일 워크로드가 GPU를 모두 사용하지 못하는 경우의 처리량을 향상시킬 수 있습니다.

다만 MPS는 시분할과 동일하게 GPU 메모리와 장애 영역을 격리하지 않습니다. 한 프로세스의 메모리 부족(Out of Memory, OOM)이나 비정상 종료가 동일 GPU를 공유하는 다른 프로세스에 직접 영향을 줄 수 있으며, 컴퓨트 슬라이스가 물리적으로 분리되지 않으므로 멀티테넌트 운영의 강한 격리 요건에는 부합하지 않습니다. 별도 라이선스 비용 없이 사용 가능하고 NVIDIA Kubernetes Device Plugin이 MPS 공유 모드를 지원하므로, 단일 사용자 또는 단일 신뢰 경계 안에서 다수의 소규모 추론·전처리 워크로드를 한 GPU에 집적해 활용률을 높이는 용도, HPC 다중 프로세스 작업의 GPU 공유 용도로 적합합니다.

• MIG(Multi-instance GPU)

MIG는 NVIDIA A100 및 H100 등 일부 데이터 센터급 GPU 아키텍처에서 지원하는 하드웨어 레벨의 분할 기술입니다. 이 기술은 단일 물리 GPU를 최대 7개의 독립 인스턴스로 나누고, 각 인스턴스마다 SM, 메모리, 메모리 대역폭을 물리적으로 분리해 할당합니다. 이러한 물리적 자원 격리는 특정 인스턴스에서 발생한 연산 부하가 다른 인스턴스의 성능에 영향을 주지 않는 수준 높은 안정성을 보장하므로 멀티테넌트 환경에 적합합니다.

그러나 강력한 격리 수준에도 불구하고 다음과 같은 몇 가지 기술적 제약사항이 있습니다. MIG의 분할 프로파일은 GPU 모델에 따라 사전 정의되어 있고 조합 규칙이 존재합니다. 예를 들어 A100 40GB 모델의 경우 7개의 1g.5gb 인스턴스 또는 3개의 2g.10gb 인스턴스를 생성하는 등 정해진 패턴 안에서만 분할이 가능합니다. 또한 운영 중 프로파일을 변경하려면 기존에 할당된 컴퓨트 인스턴스(Compute Instance, CI)와 GPU 인스턴스(GPU Instance, GI)를 해제한 뒤 재생성해야 하므로, 해당 GPU에서 실행 중인 워크로드의 종료가 수반됩니다. 따라서 MIG 기반 운영은 워크로드의 자원 요구 분포를 사전에 파악해 적절한 프로파일 조합을 결정하는 운영 설계가 선행되어야 합니다.

• vGPU(Virtual GPU)

vGPU는 NVIDIA가 가상화 환경(Virtualization)을 위해 제공하는 분할 기술로, 하이퍼바이저 계층에서 단일 물리 GPU를 다수의 가상 GPU로 분할해 개별 가상 머신에 할당하는 방식입니다. 다만 vGPU는 NVIDIA의 별도 상용 라이선스(NVIDIA vGPU Software 또는 NVIDIA AI Enterprise)와 라이선스 서버 운영을 요구합니다. 시분할은 Device Plugin 수준, MIG는 GPU 하드웨어 수준에서 별도 라이선스 비용 없이 사용 가능한 반면, vGPU는 GPU·사용자 단위 구독형 과금이 적용되므로 대규모 GPU 클러스터에서는 라이선스 비용이 운영 비용의 상당한 비중을 차지할 수 있습니다.

표 2-12 GPU 분할 방식 비교

구분	시분할	MPS	MIG	vGPU
격리 수준	시간 분할 (소프트웨어)	프로세스 단위 공간 분할(소프트웨어)	하드웨어 격리	하이퍼바이저 격리
지원 GPU	대부분의 NVIDIA GPU	CUDA 지원 NVIDIA GPU	A100, H100, H200, B100, B200 등	vGPU 인증 데이터 센터 GPU(A100, H100, L40 등 일부 모델)
분할 단위	시간 쿼텀	프로세스 단위	사전 정의 프로파일	vGPU 프로파일(메모리 단위)
런타임 재구성	가능	가능 (프로세스 재시작 수반)	인스턴스 재생성 필요 (실행 중 워크로드 종료 수반)	VM 재시작 필요
라이선스 비용	별도 비용 없음	별도 비용 없음	별도 비용 없음	별도 vGPU Software 라이선스
적합 워크로드	개발, 실험, 교육	소규모 추론, 다중 프로세스 공유	멀티테넌트 운영, 추론	가상화 환경, VDI

GPU 결합(GPU Scaling): 대규모 연산 자원 확보

단일 GPU의 처리 능력을 초과하는 워크로드, 특히 모델 크기가 단일 GPU 메모리를 초과하는 일부 대규모 모델의 학습이나 추론은 여러 GPU를 결합해 처리합니다. 결합 범위에 따라 노드 내 결합과 노드 간 결합으로 구분합니다.

• Multi-GPU Scale-up

단일 노드 내 복수의 GPU를 NVLink·NVSwitch 등 노드 내부 고속 인터커넥트로 결합합니다. NVIDIA 데이터 센터 GPU의 NVLink 양방향 대역폭(GPU 1장 기준)은 NVLink 3세대(A100) 600GB/s, NVLink 4세대(H100·H200) 900GB/s, NVLink 5세대(B100·B200, Blackwell) 1.8TB/s에 이르며, NVSwitch로 결합된 8-GPU 노드의 합산 대역폭은 수 TB/s 수준입니다. 지연도 수백 ns에서 수 마이크로초 수준으로 매우 낮아, 모델 병렬화나 텐서 병렬화 같은 GPU 간 통신이 빈번한 패턴에 적합합니다. 그러나 단일 노드의 GPU 슬롯 수에 따라 확장에 물리적 상한이 존재하므로, 그 이상의 모델은 다수 노드에 걸쳐 분산해야 합니다.

• Multi-Node Scale-out

InfiniBand나 RoCE 등 고속 네트워크로 다수 노드를 연결해 더 큰 모델을 학습하거나, 모델 크기가 단일 노드 메모리를 초과하는 일부 대규모 모델 추론을 분산 처리합니다. 노드 간 분산 학습은 노드 내 분산보다 통신 비용이 높으므로, 분산 학습 프레임워크(PyTorch 등)는 NCCL 등 집합 통신 라이브러리로 AllReduce, AllGather 등 통신 패턴을 최적화합니다. 노드 간 분산 학습 환경에서는 두 가지 추가 요구가 발생합니다.

첫째, Gang scheduling입니다. 대부분의 동기식 분산 학습은 모든 워커(Worker)가 동시에 시작해야 의미가 있습니다. N개의 워커 중 일부만 스케줄링되고 나머지가 대기 상태에 빠지면 이미 스케줄링된 GPU는 동기화 지점에서 무한 대기하며 자원만 점유하게 됩니다. Gang scheduling은 한 작업의 모든 Pod가 동시에 가용 자원을 확보할 수 있을 때만 일괄 스케줄링하는 정책입니다. Kubernetes 기본 스케줄러는 Pod 단위 스케줄링이 기본이므로, Gang scheduling을 구현하려면 Coscheduling-Volcano 등 별도 스케줄러나 플러그인이 필요합니다.

둘째, 토폴로지 인식 배치(Topology-aware Scheduling)입니다. 동일 InfiniBand 스위치 하위 노드들끼리, 또는 동일 NVLink 도메인 내 GPU들끼리 묶어 배치하면 통신 비용이 줄어듭니다. 분산 학습의 학습 시간은 통신 시간에 크게 좌우되므로, 토폴로지 인식 배치는 단순 자원 가용성 외에 물리적 인접성을 함께 고려해 워커들을 배치합니다. 이를 위해서는 클러스터의 물리 토폴로지(스위치 계층, NVLink 도메인 등)를 스케줄러가 인지할 수 있는 형태로 모델링해야 합니다.

단편화 방지를 위한 빈패킹 전략

동적 할당 환경에서는 워크로드의 생성과 종료가 반복되면서, 클러스터 합산으로는 가용 GPU가 충분하더라도 한 노드 안에서 N개의 GPU를 한 번에 확보하지 못해 작업이 스케줄링되지 못하는 단편화(Fragmentation) 상태가 발생할 수 있습니다.

이를 해결하기 위해 빈패킹(Bin Packing) 정책을 사용합니다. 빈패킹은 신규 워크로드를 가능한 한 이미 사용 중인 노드에 우선 배치해 빈 노드를 보존함으로써 후속 대규모 작업을 위한 연속 자원을 확보하는 정책입니다. 빈패킹 알고리즘에는 여러 변형이 존재합니다.

- **First-fit**: 노드를 정해진 순서로 순회하며 처음 적합한 노드에 배치하는 방식으로 구현이 단순합니다.
- **Best-fit**: 적합한 노드 중 잔여 자원이 가장 적은 노드에 배치하는 방식입니다. 단편화 측면에서 유리하지만 비교 비용이 높습니다.
- **Worst-fit**: 잔여 자원이 가장 많은 노드에 배치하는 방식으로 자원 분산이 유리한 시나리오에 적합합니다.

실제 스케줄러는 단순 알고리즘이 아니라 자원별 가중치 점수 함수로 일반화된 형태로 구현되는 경우가 많으며, 어떤 자원을 1차 기준으로 둘 것인지와 가중치를 어떻게 부여할 것인지는 클러스터 운영 정책의 결정 사항입니다.

자원 관리 정책 모델

대규모 멀티테넌트 클러스터에서는 자원의 물리적 배치 외에 정책 차원의 관리가 필요합니다. Borg-YARN 등 대규모 클러스터 운영 사례에서 도출되어 널리 채택된 모델은 다음 네 가지 요소로 구성됩니다.

- **쿼터(Quota)**: 사용자·팀·프로젝트가 사용할 수 있는 자원의 상한을 지정합니다. 신규 워크로드는 쿼터 범위 안에서만 생성될 수 있습니다.

- **보장 할당량(Guaranteed Quota)**: 특정 사용자·프로젝트가 최소한 확보해야 하는 자원의 하한을 지정합니다. 다른 사용자가 클러스터를 가득 채우고 있어도 보장 할당량 분량은 회수해서 제공할 수 있어야 합니다.
- **오버커밋(Overcommit)/오버부킹(Overbooking)**: 쿼터 합계를 물리 자원 총량보다 크게 발급하는 정책입니다. 모든 사용자가 동시에 쿼터 한도까지 사용하지 않는 통계적 특성을 활용해 클러스터 전체 활용률을 끌어올립니다.
- **회수(Reclaim)**: 유휴 상태로 판단된 자원을 자동으로 회수해 다른 워크로드에 재배분하는 메커니즘입니다.

이 네 요소는 단독이 아니라 서로 맞물려 동작합니다. 오버부킹으로 발급한 쿼터가 실제 수요와 충돌하는 순간 보장 할당량 정책이 작동해 우선순위 워크로드를 보호하고, 동시에 회수 메커니즘이 유휴 자원을 회복해 충돌 영역을 최소화 합니다.

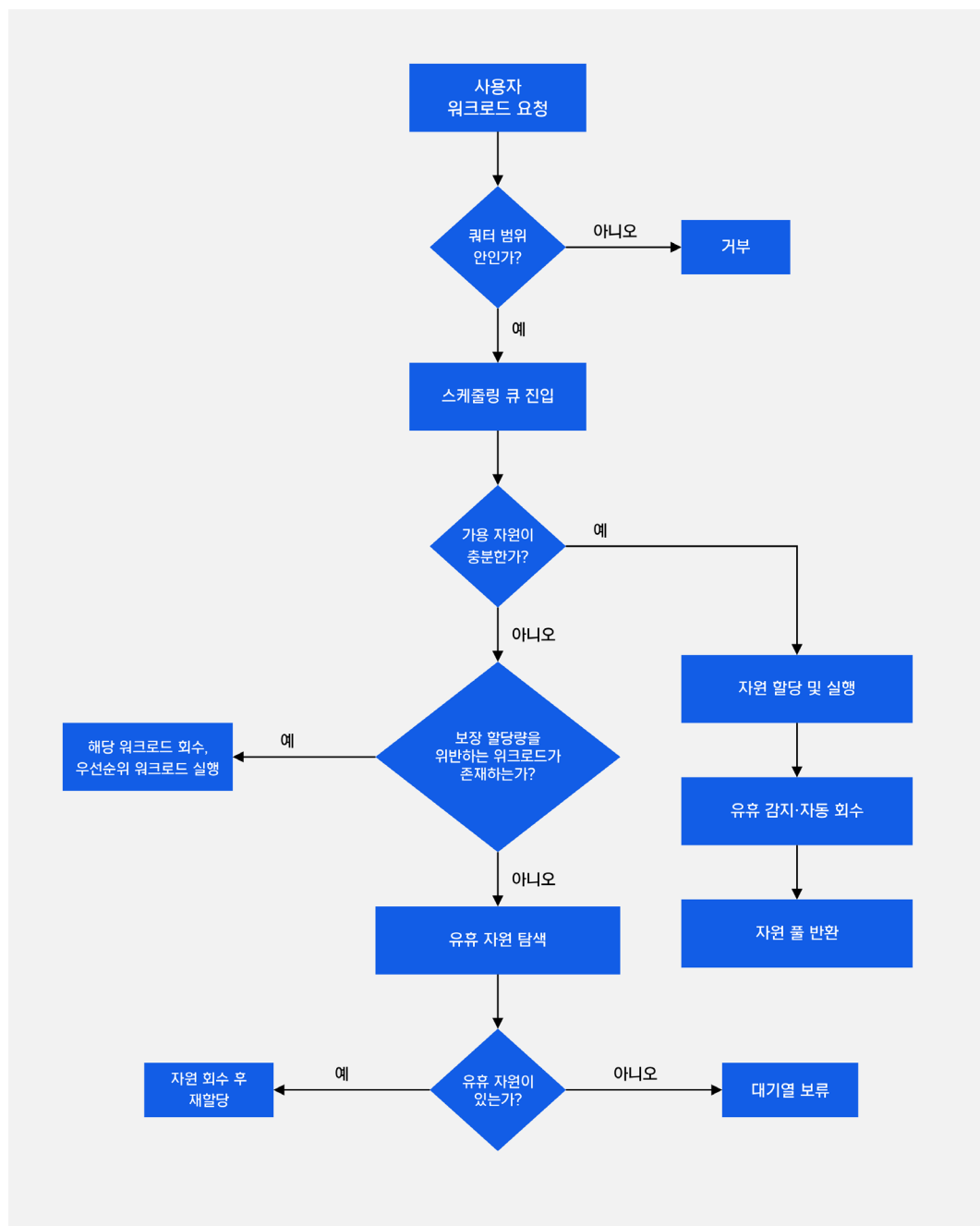


그림 2-1 자원 관리 정책 모델

도식의 좌측 흐름은 신규 워크로드 요청에 대한 쿼터·보장·회수 정책의 분기를 보여주며, 우측의 보조 흐름은 실행 중 워크로드에 대한 유휴 감지·자동 회수를 통해 자원 풀이 지속적으로 회복되는 과정을 나타냅니다. 이 모델은 Borg, YARN, Kubernetes 생태계의 Kueue·Volcano 등에 공통적으로 채택되어 있으며, 구현 세부 차이는 있으나 개념적 골격은 동일합니다.

네 요소가 동시에 작동하면서 발생하는 자원 경합은 우선순위(Priority)와 선점(Preemption)으로 조정됩니다. 회수가 유휴 상태 자원을 능동적으로 회복하는 메커니즘이라면, 선점은 활성 상태 워크로드 중 우선순위가 낮은 것을 강제로 종료해 더 높은 우선순위 작업에 자원을 즉시 제공하는 메커니즘입니다. 보장 할당량의 즉시 충족이나 시간 민감 워크로드의 서비스 수준 약정(Service Level Agreement, SLA) 보장은 유휴 회수만으로는 시점을 맞추기 어려우므로 선점이 보완적으로 작동합니다. Borg는 production-tier 작업이 batch-tier 작업을 priority 기반으로 선점하는 모델을 채택하고, YARN Capacity Scheduler는 큐 간 선점을 지원하며, Kubernetes는 PriorityClass와 Preemption을 표준 메커니즘으로 제공합니다.

자원 관리 4요소의 상호작용과 충돌 시나리오

자원 관리의 네 가지 요소가 동시에 작동할 때 발생하는 일반적인 충돌 시나리오를 통해 각 정책의 실효성을 살펴볼 수 있습니다. 예를 들어, 클러스터의 전체 물리 GPU 자원이 100개이고 프로젝트 A와 B에 각각 60개씩의 쿼터를 발급했다고 가정합니다(오버부킹 비율 1.2). 이때 A 프로젝트에는 40개의 보장 할당량이 설정되어 있고 B 프로젝트는 별도의 보장량 없이 가용 자원을 사용하는 구조입니다. 평소에는 두 프로젝트의 합산 자원 점유율이 80~90% 수준을 유지한다면 정책 간 충돌이 발생하지 않습니다.

문제는 두 프로젝트의 자원 수요가 동시에 급증하여 각자의 쿼터 한도인 총 120개에 근접할 때 발생합니다. 물리 자원은 100개에 불과하므로 누군가는 자원을 받지 못합니다. 이 시점에 보장 할당량과 회수 정책이 작동합니다. A 프로젝트가 신규 워크로드를 요청했을 때 자신의 보장 할당량(40) 이내라면, 시스템은 B 프로젝트의 회수 가능 워크로드를 선점·회수해 A 프로젝트의 보장량(40)을 우선 확보하며, A의 요청이 이를 초과할 경우 쿼터 한도(60) 범위 안에서 추가 확보를 시도합니다. 그 결과 B 프로젝트는 자신의 쿼터(60) 범위 안에서 ‘물리 자원 100에서 A의 점유량을 뺀 잔여’만큼 사용 가능하며, 충돌 시점에는 우선순위·정책에 따라 일부 워크로드가 대기열로 보류됩니다. 이러한 충돌은 빈도와 강도에 따라 운영 모니터링·정책 조정의 대상이 됩니다.

2.4.2 GPU Live의 동적 할당 기술

GPU Live는 NHN FactoryX의 GPU 자원을 단일 풀로 추상화하고, 앞서 정리한 동적 할당 모델을 Kubernetes 기반 플랫폼 위에서 통합 운영하는 자원 관리 계층입니다. GPU Live는 물리 클러스터와 실제 워크로드 실행 사이에서 자원 풀링, 정책 적용, 라이프사이클 관리를 담당하며, 사용자에게는 셀프 서비스 형태의 자원 사용 경험을 제공합니다.

Kubernetes 표준 생태계에서의 통합 운영

GPU Live는 Kubernetes 표준 생태계의 검증된 컴포넌트를 기반으로 동적 할당 플랫폼을 구성합니다. 자체 스케줄러나 클러스터 매니저를 새로 만드는 대신, Kubernetes 표준 인터페이스(Device Plugin, Topology Manager 등)와 그

위에서 동작하는 GPU 워크로드용 스케줄러를 활용해 동적 할당의 핵심 기능을 구현합니다. 이러한 접근은 다음 이점을 가집니다.

- **표준성:** Kubernetes 생태계의 표준 인터페이스를 따르므로 GPU 디바이스 드라이버 업데이트, 분산 학습 프레임워크 변화 등 외부 변동을 표준 경로로 흡수할 수 있습니다.
- **운영 검증성:** 검증된 오픈소스 소프트웨어(Open Source Software, 이하 ‘OSS’) 컴포넌트는 글로벌 사용자 기반으로 운영상 안정성이 확보된 상태이므로, 자체 구현 대비 초기 위험이 낮습니다.
- **확장성:** 표준 인터페이스를 따르는 한 향후 새로운 가속기(NPU 등) 또는 새로운 워크로드 패턴이 등장해도 표준 경로를 통해 통합이 가능합니다.

GPU Live의 차별점은 어느 OSS 컴포넌트를 사용했느냐가 아니라, 다수의 표준 컴포넌트를 어떻게 통합하고 어떤 운영 정책을 그 위에 정의했느냐에 있습니다. 멀티테넌시, 자원 관리, 라이프사이클 관리는 GPU Live가 직접 설계한 영역으로, Kubernetes 표준 위에서 NHN FactoryX 환경에 맞춘 운영 모델을 구현합니다.

멀티테넌시 계층 구조

GPU Live는 다수의 조직과 프로젝트가 단일 GPU 풀을 공유하면서도 자원·정책·접근 권한이 격리되도록 3계층 멀티테넌시 모델을 운영합니다. 시스템(System) 계층은 클러스터 운영자가 관리하는 최상위 계층이며, 그 아래에 조직(Organization) 계층, 다시 그 아래에 프로젝트(Project) 계층이 위치합니다. 각 계층은 자원 쿼터, 멤버 권한, 워크로드 가시성을 독립적으로 가집니다.

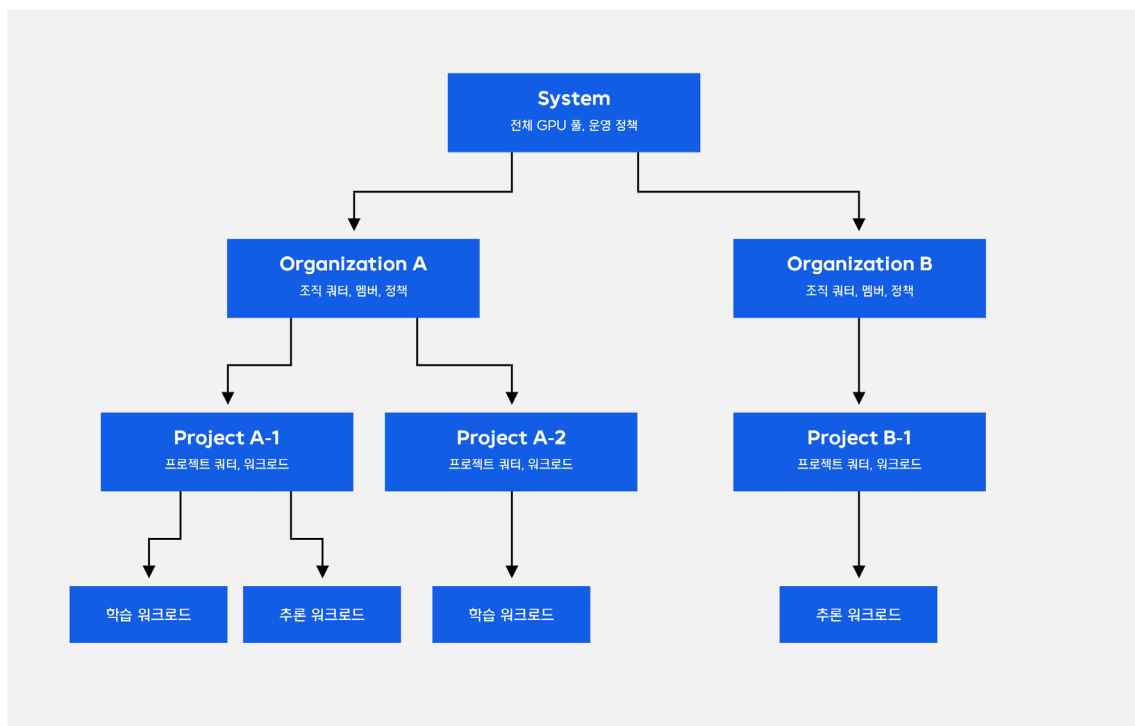


그림 2-2 멀티테넌시 계층 구조

자원 격리는 조직과 프로젝트가 각각 GPU 쿼터를 보유하는 구조로 구현됩니다. 워크로드 생성 시점에는 프로젝트 쿼터를 검증해 한도를 초과하는 요청을 차단하며, 실시간 자원 사용량은 프로젝트 단위로 갱신되어 그 합산값이 조직 사용량으로 자동 누적됩니다. 이로써 조직·프로젝트 단위 사용 현황이 분리되어 추적·관리됩니다.

정책 상속은 시스템 운영자나 조직 관리자가 단일 변경만으로 하위 계층 전체에 정책을 일괄 적용할 수 있도록 합니다. 예를 들어 시스템 계층에서 정의한 기본 자원 회수 임계값은 정책 상속으로 모든 조직과 프로젝트에 자동 적용되며, 특정 조직이나 프로젝트가 별도 정책을 정의한 경우 하위 정의가 상위 정의를 부분 재정의(override)합니다.

접근 권한 격리는 역할 기반 접근 제어(Role-Based Access Control, RBAC)를 기반으로 구현됩니다. 각 프로젝트는 Kubernetes 네임스페이스로 매핑되고, 프로젝트 멤버에게는 해당 네임스페이스 범위의 권한만 부여됩니다. 조직 수준 권한은 조직 내 모든 프로젝트 네임스페이스에 대한 권한으로 매핑되며, 시스템 운영자는 클러스터 수준 권한을 가집니다. 이러한 권한 모델을 통해 한 프로젝트의 사용자는 다른 프로젝트의 워크로드를 조회·조작할 수 없도록 격리됩니다.

이 세 메커니즘은 결합되어 다음과 같은 효과를 만듭니다. 다수의 조직이 단일 GPU 클러스터를 공유하더라도 (a) 조직 사이의 자원 사용량은 분리되어 측정·관리되며, (b) 운영 정책은 계층적으로 적용되어 운영 부담이 분산되고, (c) 접근 권한은 프로젝트 경계로 엄격히 격리되어 보안과 독립성이 유지됩니다.

GPU Live의 동적 할당 모델

GPU Live는 GPU 자원을 노드별 고정 배정이 아닌 단일 풀로 관리하며, 워크로드 요청 시점에 자원을 동적으로 할당합니다. 할당 단위는 단일 GPU에서 다수 노드의 다수 GPU에 이르기까지 워크로드 요구에 따라 가변적이며, 학습 워크로드의 Multi-GPU Scale-up과 Multi-Node Scale-out을 모두 지원합니다. 노드 간 분산 학습 환경에서는 노드 간 고속 네트워크(InfiniBand 등)로 워커들이 통신합니다.

워크로드의 동적 할당 라이프사이클은 다음과 같이 진행됩니다.

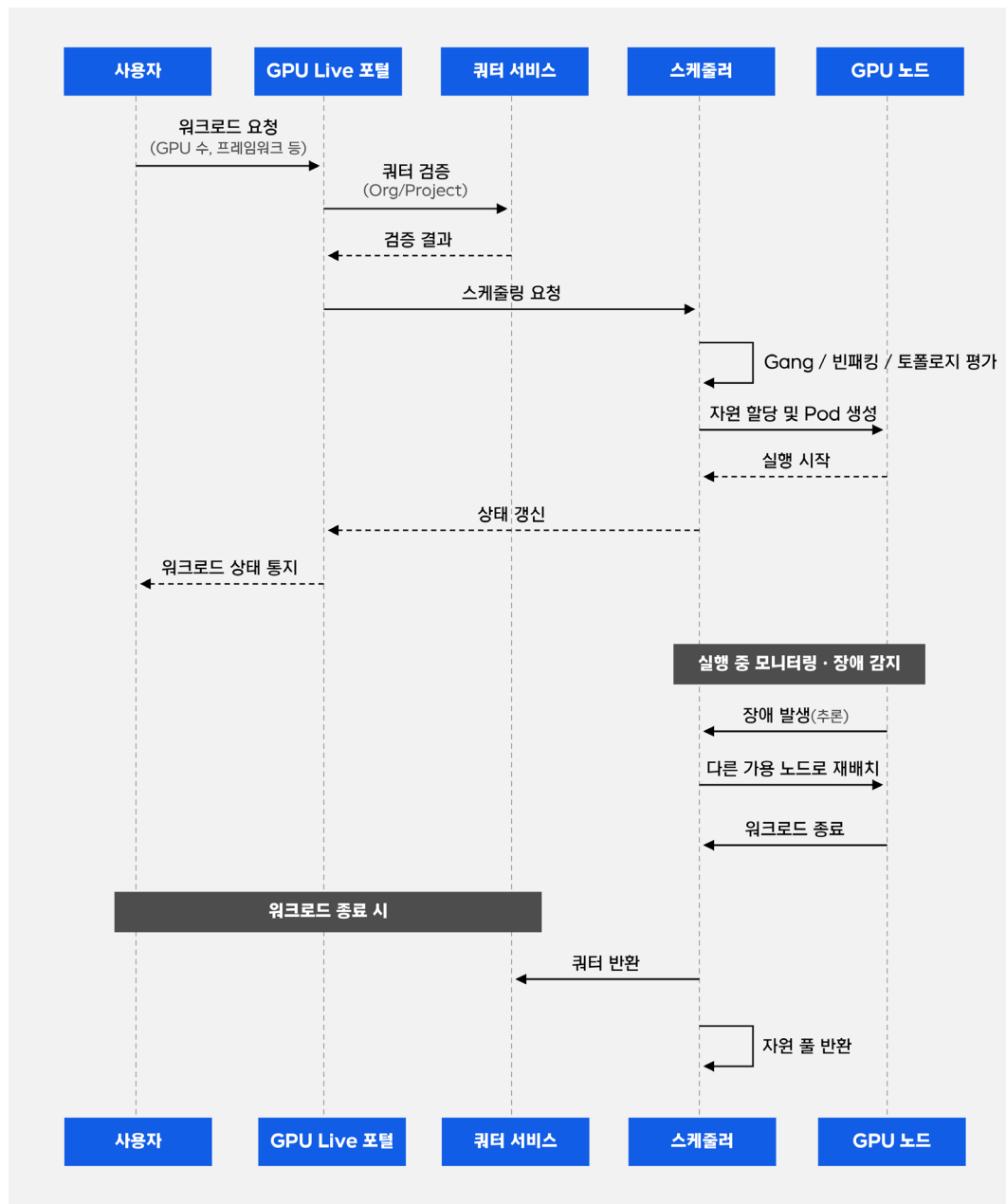


그림 2-3 GPU Live 동적 할당 라이프사이클

이 흐름은 사용자 요청에서 자원 회수까지의 전체 단계를 자동화하므로, 사용자는 자원 위치나 노드 식별을 신경 쓰지 않고 워크로드 명세에만 집중할 수 있습니다.

학습과 추론 워크로드의 라이프사이클 차이

GPU Live는 학습과 추론을 동일한 자원 풀에서 처리하면서도, 두 워크로드 유형의 성격 차이를 라이프사이클 모델에 반영합니다.

학습 워크로드는 일반적으로 다음과 같은 특성을 가집니다. 워크로드의 시작과 종료 시점이 명확하고, 학습이 완료되면 자원이 즉시 회수됩니다. 분산 학습의 경우 다수 워커가 동시에 시작·종료해야 하며, 중간에 한 워커가 실패하면 일반적으로 전체 작업의 재시작이 필요합니다. 자원 점유 시간은 작업 규모에 따라 수 시간에서 수 일까지 분포합니다.

추론 워크로드는 다른 특성을 가집니다. 워크로드는 트래픽이 발생하는 동안 지속적으로 실행되는 장기 서비스 형태이

며, 종료 시점이 사전에 정해지지 않습니다. 부하가 변동되므로 인스턴스 수가 동적으로 변할 수 있고, 인스턴스 단위 장애가 전체 서비스 중단으로 이어지지 않도록 인스턴스 단위 회복·재배치가 요구됩니다.

GPU Live는 이러한 차이를 라이프사이클 모델에 다음과 같이 반영합니다.

- **학습:** Gang scheduling으로 일괄 시작하고, 작업 완료 시 일괄 종료하며 자원을 회수합니다. 워커 일부 실패 시 전체 작업 재시작 또는 사용자 정의 재시도 정책이 적용됩니다.
- **추론:** 인스턴스 단위 GPU 세트 할당, 인스턴스 라이프사이클이 종료될 때까지 자원을 유지합니다. 노드 또는 Pod 장애 발생 시 다른 가용 노드로 재배치하는 정책을 운영하며, 재배치 시점에 쿼터 재검증과 빈패킹 정책이 다시 적용됩니다.

추론 워크로드의 부하 변동 대응(예: 트래픽 증가에 따른 인스턴스 수평 확장)은 Kubernetes 표준 자동 스케일링 메커니즘으로 처리되며, GPU Live는 그 위에 자원 관리·정책·격리 계층을 제공합니다.

멀티 노드 스케줄링

멀티 노드로 생성하는 워크로드에 대해 GPU Live는 Gang scheduling을 적용합니다. 작업에 필요한 모든 워커가 동시에 가용한 상태가 아니면 작업 전체가 대기열에 보류되어, 부분 시작으로 인한 자원 낭비와 동기화 무한 대기를 구조적으로 차단합니다. 클러스터 부하가 높은 환경에서는 작업 대기 시간이 늘어날 수 있으나, 이는 빈패킹 정책 및 자원 회수 정책으로 완화할 수 있습니다.

빈패킹 스케줄링

GPU Live는 GPU를 우선 기준으로 한 Best-fit 정책으로 빈패킹을 운영합니다. 신규 워크로드는 가능한 한 이미 GPU가 사용 중인 노드에 우선 배치되며, 그 결과 빈 노드가 보존되어 후속 분산 학습 등 대규모 작업을 위한 연속 자원으로 활용됩니다. 이러한 GPU 중심 빈패킹은 다음 두 효과를 동시에 달성합니다.

첫째, 노드 단위 GPU 단편화를 최소화합니다. 동일한 워크로드 부하라도 빈패킹 적용 여부에 따라 빈 노드 수가 크게 달라집니다. 빈패킹이 없으면 워크로드가 다수 노드에 분산 배치되어 어느 노드도 완전히 비지 않는 상황이 흔하지만, 빈패킹은 사용 중 노드부터 채움으로써 빈 노드가 결과적으로 보존됩니다.

둘째, 대규모 작업의 가용성을 확보합니다. 분산 학습 작업은 일반적으로 동일 노드 내에서 다수 GPU를 한 번에 확보해야 하므로, 보존된 빈 노드가 곧 분산 학습 작업의 가용 자원이 됩니다. 빈패킹 없이 클러스터에 가용 GPU가 충분히 많아도 분산 학습 작업이 대기열에 머무는 상황이 빈패킹을 통해 완화됩니다.

자원 관리

GPU Live는 2.4.1에서 정리한 [자원 관리 정책 모델](#) 가운데 현재 운영 단계에서 쿼터·오버부킹·회수의 세 요소를 통합 운영하고 있습니다. 보장 할당량은 운영 모델의 다음 단계 도입 항목으로, 현재는 미적용 상태입니다.⁴

4 2.4.3 향후 개발 방향 참조

- **쿼터 기반 스케줄링**

조직과 프로젝트 두 계층에 GPU 쿼터를 정의하고, 워크로드 생성 시 프로젝트 쿼터를 검증해 한도를 초과하는 생성 요청을 차단합니다. 조직 쿼터는 사용 현황 추적·정책 적용 단위로 활용되며, 조직 단위 강제 상한은 보장 할당량 도입과 함께 단계적으로 강화될 예정입니다.

- **오버부킹(Overbooking)**

사용자 쿼터 총합을 물리 GPU 총량보다 크게 발급해 클러스터 전체 활용률을 향상시키는 정책입니다. 모든 사용자가 동시에 쿼터 한도까지 사용하지 않는 통계적 특성을 활용하며, 클러스터 부하가 높은 시점에는 회수 메커니즘과 결합해 자원 충돌을 자동 해소합니다.

- **유휴 자원 자동 회수**

워크로드 정상 종료 시점의 즉시 회수, 일정 시간 이상 GPU 사용량이 임계값 이하인 워크로드의 정책 기반 회수, 프로젝트 만료·자원 정책 위반 시점의 강제 회수가 모두 자동 수행됩니다. 회수된 자원은 즉시 풀로 반환되어 다른 워크로드의 스케줄링에 사용됩니다.

GPU Live 환경의 충돌 시나리오 — 운영 관점

쿼터·오버부킹·회수가 동시에 작동할 때 GPU Live는 다음과 같이 처리합니다. 클러스터 GPU 풀의 사용률이 90%를 초과하고, 한 프로젝트가 신규 워크로드를 요청하는 상황을 가정합니다.

1. 프로젝트의 쿼터 검증을 수행한다. 쿼터를 초과하면 즉시 거부한다.
2. 쿼터 범위 내 요청이라면 가용 자원 탐색을 시도한다.
3. 가용 자원이 부족한 경우 ‘회수 가능(reclaimable)’으로 표시된 워크로드 후보를 선별한다. 회수 후보는 일정 시간 이상 GPU 사용량이 임계값 이하인 유휴 워크로드, 우선순위가 낮은 회수 가능 워크로드 등이다.
4. 회수 대상 워크로드를 회수해 자원을 확보한 뒤 신규 워크로드를 스케줄링한다. 회수된 워크로드는 자원 가용 시점에 다시 스케줄링되도록 큐로 복귀한다.
5. 이 모든 과정이 자동으로 진행되며, 운영자는 회수·재배치 이력을 모니터링 도구로 추적할 수 있다.

이러한 처리는 오버부킹으로 발급한 쿼터 합계가 실제 사용 시점에 물리 자원 한계와 충돌할 때 회수 메커니즘으로 충돌을 해소하는 일반적인 동작입니다. 결과적으로 클러스터 활용률은 오버부킹 정책으로 높게 유지되면서도, 충돌 시점에는 회수가 자동으로 작동해 신규 워크로드의 스케줄링 가능성을 확보합니다. 사용자·프로젝트 단위 SLA를 정량적으로 보장하는 단계는 보장 할당량 도입과 함께 단계적으로 강화될 예정입니다.

2.4.3 향후 개발 방향⁵

본 절에서는 GPU 자원의 동적 할당 일반 기술과 GPU Live의 운영 모델을 살펴보았습니다. 일반 기술 관점에서 동적 할당은 GPU 분할, GPU 결합, 빈패킹, 그리고 쿼터·보장 할당량·오버부킹·회수의 네 요소로 구성된 자원 관리 모델을 통해 정적 할당의 활용률 한계를 극복합니다. GPU Live는 이러한 일반 기술을 Kubernetes 표준 생태계 위에서 통합 운영하며,

⁵ 2026년 6월 시점 기준

3계층 멀티테넌시(System, Organization, Project), GPU를 우선 기준으로 한 빈패킹, 쿼터·오버부킹·회수의 통합 운영, 학습·추론 워크로드별 라이프사이클을 차별화해 단일 GPU 풀에서 다양한 AI 워크로드를 안정적으로 수용합니다.

GPU 동적 할당 분야는 Kubernetes 표준 자체도 빠르게 진화하고 있습니다. 대표적으로 Kubernetes 1.34에서 GA에 진입한 동적 자원 할당(Dynamic Resource Allocation, DRA)은 기존 Device Plugin 모델을 대체할 차세대 디바이스 자원 추상화로, 동적 자원 청구·구조화된 자원 모델·다중 가속기 통합을 표준화합니다. NVIDIA, AMD, Intel 등 주요 가속기 벤더가 DRA 기반 드라이버로 전환하고 있으며, 토폴로지 인식·MIG·이종 가속기 통합 운영의 표준 경로로 자리 잡고 있습니다. GPU Live 역시 Device Plugin 기반의 현재 운영을 유지하면서 DRA 도입을 단계적으로 검토할 예정입니다.

GPU Live는 이러한 표준 진화를 흡수하면서 동적 할당 모델의 기능 고도화와 적용 범위 확장을 단계적으로 진행하고 있습니다. 기능 측면에서는 단일 GPU 세분화를 위한 공간 분할(MIG) 지원, 노드 내 NUMA(non-uniform memory access) affinity와 다중 노드 간 토폴로지 인식 배치, 학습·추론 동시 배치(Co-location) 정책 정교화를 통한 단일 노드 활용률 제고, 우선순위 프로젝트의 SLA 정량 보장을 위한 보장 할당량 도입이 진행되고 있습니다. 적용 범위 측면에서는 GPU와 동일한 동적 할당 모델을 NPU 등 이종 가속기로 확장하기 위한 기술 검토가 이루어지고 있으며, 국내 NPU가 추론에 특화된 점을 고려해 추론 워크로드를 1차 적용 대상으로 삼고 학습 가능한 NPU 제품군 확대에 맞춰 학습 워크로드까지 점진적으로 넓혀갈 예정입니다. 디바이스 인식·할당 인프라는 이미 운영 환경에 도입되어 있으며, 스케줄링과 자원 관리 정책의 통합은 단계적으로 진행됩니다. 이러한 확장은 단일 운영 모델로 다양한 AI 가속기와 워크로드를 통합한다는 점에서, 향후 AI 인프라 다양화에 대한 대응 기반이 될 것입니다.

03 하드웨어 구성

- 3.1 CPU 및 메모리 구성
- 3.2 GPU 사양 및 성능
- 3.3 스토리지 및 네트워크

03 하드웨어 구성

하드웨어 구성은 단일 노드의 사양 합산이 아닌, 클러스터 전체를 하나의 시스템으로 보았을 때의 균형을 우선 기준으로 설계됩니다. 본 장에서는 NHN FactoryX의 GPU 노드 표준 구성(Reference Configuration)을 중심으로, 각 구성 요소의 선택 근거를 설명합니다.

3.1 CPU 및 메모리 구성

AI 워크로드에서 CPU는 모델 자체 연산이 아니라, GPU 데이터를 끊임 없이 공급(Feeding)하는 역할을 담당합니다. CPU가 부족하면 GPU가 데이터 대기 상태로 유휴(idle)에 빠지며, 이 경우 GPU 활용률이 60% 이하로 떨어지는 현상이 흔히 관측됩니다. 따라서 CPU와 메모리는 다음 원칙을 따릅니다.

- **PCIe Gen5 레인 확보:** GPU, NIC, NVMe(non-volatile memory express)가 모두 PCIe Gen5x16을 요구하므로 단일 소켓당 충분한 레인을 제공하는 최신 세대 CPU를 선택합니다.
- **NUMA 인지 설계:** GPU, NIC, NVMe가 동일 NUMA(non-uniform memory access) 노드에 정렬되도록 슬롯 배치를 표준화합니다. 동일 NUMA 도메인 안에서 데이터가 처리되면 소켓 간 인터커넥트를 경유하는 크로스 소켓 트래픽이 줄고, 학습 단계당 데이터 로딩 지연이 안정화됩니다.
- **GPU 메모리 대비 시스템 메모리 비율:** 일반적으로 시스템 메모리는 노드 내 GPU 메모리 총합의 2배 이상을 권장합니다. 데이터 프리페치, 옵티마이저 상태 오프로딩, 체크포인트 스테이징 버퍼, 분산 학습용 통신 버퍼 등 GPU 외부에서 일시적으로 유지되어야 하는 데이터의 총량이 이 수준에 근접하기 때문입니다.

다음은 AI 워크로드를 위한 CPU 및 메모리 권장 구성 예시입니다.

표 3-1 AI 워크로드 CPU/메모리 권장 구성

CPU 아키텍처	Intel Xeon Scalable(Sapphire Rapids/Emerald Rapids) 또는 AMD EPYC(Genoa/Bergamo) 듀얼 소켓 구성
코어 수	단일 소켓당 충분한 레인을 제공하는 최신 세대 CPU (예: 듀얼 소켓×56C=112C)
시스템 메모리	노드당 2TB DDR5 ECC RDIMM(구성에 따라 1TB~4TB)
메모리 채널	소켓당 8채널 이상, NUMA-aware 풀 채널 구성
PCIe	Gen5, GPU/NIC/NVMe 슬롯에 충분한 x16 레인 분배

3.2 GPU 사양 및 성능

AI 워크로드는 단계에 따라 요구되는 GPU 특성이 크게 다릅니다. 사전 학습(Pre-training)은 대용량 HBM(high bandwidth memory)과 높은 BF16/FP8 처리량, 노드 간 광대역 통신을 요구하는 반면, 추론과 파인튜닝(Fine-tuning)은 단일 모델을 다수 사용자에게 제공하는 응답 지연 및 토큰당 비용이 더 결정적인 변수입니다. 모든 워크로드에 동일한 GPU를 일괄 배정하는 방식은 비용이나 전력 효율 측면에서 비효율적이기 때문에, NHN FactoryX는 워크로드 특성에 따라 학습용 고성능 GPU 라인과 추론·파인튜닝용 GPU 라인으로 구분하여 운영합니다.

다음 표는 두 라인에 사용되는 NVIDIA GPU의 세대별 사양을 비교한 것입니다.

표 3-2 NVIDIA GPU 세대별 주요 사양 비교

항목	A100 SXM	H100 SXM	H200 SXM	B200 SXM	B300 SXM
아키텍처	Ampere	Hopper	Hopper	Blackwell	Blackwell Ultra
GPU당 HBM	80GB	80GB	141GB	180GB	270GB
HBM 대역폭	2TB/s	3.35TB/s	4.8TB/s	7.7TB/s	7.7TB/s
FP64 Tensor	19.5TFLOPS	67TFLOPS	67TFLOPS	37TFLOPS	1.25TFLOPS
TF32 Tensor	312TFLOPS	989TFLOPS	989TFLOPS	2.2PFLOPS	2.2PFLOPS
BF16/FP16 Tensor	624TFLOPS	1,979TFLOPS	1,979TFLOPS	4.5PFLOPS	4.5PFLOPS
FP8 Tensor	미지원	3,958TFLOPS	3,958TFLOPS	9PFLOPS	9PFLOPS
FP4 Tensor	미지원	미지원	미지원	18PFLOPS	18PFLOPS
NVLink 대역폭	600GB/s	900GB/s	900GB/s	1,800GB/s	1,800GB/s
GPU TDP	400W	700W	700W	1,000W	1,400W
주력 연산	FP16/BF16	FP64/FP16	FP64/FP16	FP8	FP4
대표 워크로드	범용 AI 학습	과학 계산, LLM 학습	과학 계산, 대형 모델 추론	LLM 학습·추론	LLM 추론 서비스

3.3 스토리지 및 네트워크

AI 학습 및 추론 인프라에서 스토리지와 네트워크는 GPU에 데이터를 끊임 없이 공급하는 한 쌍의 경로입니다. 학습 데이터 로드와 체크포인트 I/O는 스토리지 계층에서 시작되고, 노드 간 통신은 네트워크 패브릭을 통해 분산됩니다. 두 경로 중 어느 한쪽이라도 GPU 처리량을 따라가지 못하면 클러스터 전체의 활용률이 떨어집니다. 본 절에서는 워크로드 특성에 맞춰 분리된 스토리지 계층 구조와 트래픽 종류별로 물리적으로 분리된 네트워크 패브릭 설계를 차례대로 다룹니다.

3.3.1 스토리지 계층(Tiered Storage)

AI 학습 파이프라인의 스토리지는 단일 계층이 아닌, I/O 특성이 다른 워크로드를 각각의 계층으로 분리하여 비용 및 성능을 최적화합니다.

표 3-3 AI 학습 워크로드 스토리지 계층 구조

계층	매체	주요 용도	핵심 요구 사항
Tier 0	노드 내부 NVMe	데이터 로더 캐시, 체크포인트 스테이징	GPUDirect Storage 지원
Tier 1	병렬 파일 시스템(Lustre/GPFS/WekaFS 등)	활성 학습 데이터셋	다중 클라이언트 동시 처리량, 메타데이터 성능
Tier 2	오브젝트 스토리지	원본 데이터셋, 아카이브, 콜드 체크포인트	용량·내구성·비용 효율

세 계층은 데이터 수명 주기에 따라 협력합니다. 원본 데이터셋은 Tier 2에 보관하고, 학습 시작 시 활성 학습 데이터셋만 Tier 1으로 프리페치하며, 학습 중 데이터 로더가 사용하는 미니 배치는 Tier 0의 NVMe 캐시에 스테이징되어 GPU와 가장 짧은 경로로 연결됩니다. 체크포인트도 동일한 흐름의 역방향으로 학습 중 빈번한 임시 체크포인트는 Tier 0에 쓰고, 영구 보관용은 Tier 2로 비동기 복제합니다. 이러한 계층 분리로 GPU 인근 매체에는 핫 데이터만 두고, 콜드 데이터는 저비용 매체에 집중시킬 수 있습니다.

3.3.2 네트워크 구성

네트워크는 앞 장에서 설명한 세 패브릭(Compute, Storage, Management/In-band Fabric)을 물리적으로 분리하고, BMC/IPMI 관리를 위한 Out-of-band 평면을 별도로 운영합니다. 각 평면이 자신의 트래픽 특성에 맞춰 독립적으로 튜닝되며, 각 패브릭이 서로의 트래픽에 영향을 주지 않도록 설계됩니다.

- **Compute Fabric:** InfiniBand(인피니밴드) NDR 400Gbps×8NIC/노드(Rail-Optimized)
- **Storage Fabric:** 100/200/400GbE 또는 별도 InfiniBand 평면, GPUDirect Storage 활성화
- **Management/In-band Fabric:** 이중화된 10/25GbE, 오케스트레이션·모니터링·이미지 배포 트래픽 전용
- **Out-of-band Management:** BMC/IPMI 별도 1GbE 평면, 컴퓨팅 평면 장애와 무관한 전원·콘솔 제어 보장

본 장에서는 NHN FactoryX GPU 노드의 CPU·메모리, GPU, 스토리지, 네트워크 구성 기준과 선택 근거를 살펴보았습니다. 이어지는 [4장](#)에서는 이러한 하드웨어 구성의 이론적 연산 성능을 장비 공식 사양 기준으로 산출하고, 실제 운영 시 나타나는 성능 저하 패턴을 분석하며, 패턴별 설계 대응 체크리스트를 제시합니다.

04 성능 분석

- 4.1 이론적 연산 성능
- 4.2 성능 저하 패턴 분석
- 4.3 패턴별 설계 단계 대응 체크리스트

04 성능 분석

성능은 두 가지 축에서 평가되어야 합니다. 첫째는 장비 공식 사양 기준의 이론적 연산 성능이며, 둘째는 실제 분산 학습 및 추론 워크로드에서 달성되는 유효 성능(Effective Performance)입니다. 두 값 사이에는 거의 항상 일정한 격차가 존재하며, 이 격차의 크기를 결정하는 것이 클러스터의 실제 운영 효율입니다. 본 장에서는 먼저 이론적 연산 성능을 산출하는 방식을 정리하고, 이론치와 실측치 사이에서 격차를 만들어내는 5가지 저하 패턴과 각 패턴의 측정 지표·대응 방향을 정리한 뒤, 운영 단계가 아닌 설계 단계에서 사전에 검증해야 할 체크리스트로 마무리합니다.

4.1 이론적 연산 성능

장비 공식 사양 기준의 이론적 연산 성능은 다음 공식으로 산출됩니다.

$$\text{Cluster Peak FLOPS} = (\text{노드 수}) \times (\text{노드당 GPU 수}) \times (\text{GPU 단위 Peak FLOPS})$$

예를 들어 H100 SXM 8장을 탑재한 노드를 32대 구성한 경우, 이론적 FP8 Sparse 연산 성능은 다음과 같이 산정됩니다.

$$32 \text{ 노드} \times 8 \text{ GPU} \times 3,958 \text{TFLOPS} = \text{약 } 1.01 \text{ExaFLOPS (FP8 Sparse, 이론치)}$$

이 값은 클러스터 성능의 상한선이며, 실제 학습 워크로드의 유효 성능은 일반적으로 이 값의 40~60% 수준에서 형성됩니다. 이 비율을 모델 FLOPS 활용률(Model FLOPS Utilization, 이하 ‘MFU’)이라 부르며, 분산 학습 과정에서 GPU가 통신·데이터 대기·체크포인트 등으로 휴지하는 시간을 모두 포함한 효율 지표입니다. 잘 설계된 LLM 학습 클러스터에서는 50% 이상의 MFU를 목표로 하며, 이 수치를 달성하지 못한다면 다음 4.2에서 다룰 5가지 성능 저하 패턴 중 하나 이상이 누적되고 있을 가능성이 큼니다.

4.2 성능 저하 패턴 분석

이론적 연산 성능과 실측 성능 사이의 격차는 거의 항상 다음 다섯 가지 패턴 중 하나 또는 그 조합에서 발생합니다. 각 패턴은 측정 가능한 지표와 명확한 대응 방안을 갖습니다.

4.2.1 네트워크 병목(Communication-bound)

네트워크 병목은 GPU 연산 자원이 남아 있음에도 노드 간 통신이 따라가지 못해 GPU가 대기하는 상태를 가리킵니다. 모니터링상으로는 GPU 활용률(SM Activity)은 낮으면서 GPU 전력 소비도 평소보다 낮게 측정되고, NIC(network interface card) 트래픽 그래프에 주기적인 버스트 패턴이 보입니다. 전역 동기화(All-Reduce)가 학습 Step 전체 시간의 30% 이상을 차지하는 경우는 네트워크 병목을 의심해야 하는 대표적 징후입니다. 원인은 통신 경로 어딘가에서 처리량이 GPU 연산 속도를 따라가지 못하는 데 있으며, Oversubscribed 토폴로지나 Rail 정렬 실패 문제일 수도, NIC 당 대역폭 부족 같은 물리 자원의 문제일 수도, NCCL(NVIDIA Collective Communications Library) 환경 변수 미튜닝처럼 소프트웨어 설정의 문제일 수도 있습니다. 진단 시에는 NCCL all_reduce 시간, GPU SM Activity 대비 DRAM Active 비율, NIC RX/TX 대비 GPU FLOPS 비율을 함께 관찰하여 어느 단계에서 처리량이 정체되고 있는지를 확인합니다.

4.2.2 메모리 부족(HBM/시스템 메모리)

메모리 부족은 OOM(out of memory) 오류로 직접 드러나거나, OOM을 피하려고 한 번에 처리하는 데이터 묶음(마이크로배치)을 강제로 줄이면서 학습 속도가 떨어지는 형태로 나타납니다. 메모리 압박이 더 심해지면 활성화 재연산(Activation Recomputation)이 쓰입니다. 계산 중간 결과값을 메모리에 저장하지 않고 버린 뒤, 필요할 때 다시 계산해 메모리를 아끼는 방식입니다. 대신 같은 계산을 반복하므로 학습 속도가 그만큼 느려집니다. 근본 원인은 모델 파라미터, 옵티마이저 상태, 중간 결과값(활성화)의 합이 GPU의 HBM(High Bandwidth Memory) 용량을 초과하거나, 시스템 메모리가 부족해 디스크와 메모리 사이를 오가는 지연(페이지 폴트)이 자주 발생하는 데 있습니다. 진단 시에는 GPU 메모리 점유율의 피크값, OOM 발생 빈도, 활성화 재연산 적용 비율을 함께 추적하여 메모리 압박이 어느 정도까지 누적되었는지 확인합니다.

4.2.3 스토리지 I/O 병목

스토리지 I/O 병목은 에포크(Epoch) 시작부터 GPU 활용률이 0에 가깝게 떨어지고, 시간이 지나도 회복되지 않는 형태로 나타납니다. GPU가 처리할 데이터를 공급받지 못해, 데이터 로더 워커가 계속 대기 상태에 머무릅니다. 원인은 대체로 네 가지입니다. 스토리지 자체의 처리량이 부족하거나, 작은 파일이 너무 많아 파일을 찾고 여는 과정(메타데이터 처리)에서 병목이 생기거나, GPUDirect Storage가 꺼져 있어 데이터가 CPU를 거치는 느린 경로로 이동하거나, 데이터를 읽어들이는 로더 워커 수가 부족한 경우입니다. 실제 현장에서는 이 가운데 둘 이상이 겹쳐 나타나는 경우가 많습니다. 진단 시에는 Step 시간 대비 데이터 로더 대기 시간 비율, 스토리지 처리량(GB/s), 초당 메타데이터 오퍼레이션 수를 함께 측정하여, 병목이 처리량 부족 때문인지 메타데이터 처리 때문인지 구분합니다.

4.2.4 열 스로틀링(Thermal Throttling)

열 스로틀링은 장시간 학습이 진행됨에 따라 GPU 클럭이 단계적으로 하락하고, 동일한 학습 Step의 소요 시간이 처음 보다 점진적으로 증가하는 형태로 나타납니다. 원인은 랙 단위 냉각 용량의 부족, Hot/Cold Aisle 격리 미흡, 공조 풍량 불균형, 그리고 클러스터를 동시에 풀로드로 가동했을 때의 열 부하를 사전에 예측하지 못한 데 있습니다. 진단 시에는 랙·노드·소켓별 GPU 온도 분포, 클럭 스로틀링 이벤트 발생률, 동일 워크로드를 수행한 노드 간 처리량 편차를 함께 비교하여 어느 위치에서 열적 한계가 누적되고 있는지를 추적합니다.

4.2.5 체크포인트 오버헤드

체크포인트 오버헤드는 정기 체크포인트를 저장하는 시점마다 모든 GPU가 수십 초에서 수 분간 유힤(idle) 상태에 빠지는 형태로 나타납니다. 학습은 멈춰 있고 저장만 진행되는 구간으로, 대규모 모델에서는 한 번 저장하는 데에만 분 단위 시간이 걸립니다. 원인은 대체로 네 가지입니다. 한 노드에서 순차적으로(직렬) 저장하는 구조이거나, 스토리지 쓰기 대역폭이 부족하거나, 학습과 저장을 동시에 진행하는 비동기 체크포인트가 적용되지 않았거나, 보존 가치에 비해 저장 주기가 지나치게 잦은 경우입니다. 진단 시에는 체크포인트 1회 저장 소요 시간, 학습 시간 대비 체크포인트 누적 시간 비율, 장애 복구(restore) 소요 시간을 함께 추적하여, 저장과 복구 양쪽에서 시간 비용이 얼마나 누적되는지 확인합니다.

4.3 패턴별 설계 단계 대응 체크리스트

운영 단계에서 위 패턴이 발견되면 이미 학습 비용과 일정에 영향이 발생한 상태이며, 사후 대응은 워크로드 정지나 클러스터 재구성 같은 큰 비용을 동반합니다. 따라서 다음 체크리스트는 클러스터 설계 단계에서 각 패턴을 사전 차단하기 위해 검증해야 할 항목으로, 4.2에서 다룬 다섯 가지 패턴과 1:1로 대응합니다.

표 4-1 성능 저하 패턴별 설계 단계 체크리스트

네트워크 병목	• Leaf-Spine 1:1 비차단 검증 • Rail-Optimized 배선 검증 • GPU당 NIC 대역폭 비율($\geq 100\text{Gbps/GPU}$ 권장) • SHARP·Adaptive Routing 활성화 • NCCL 토폴로지 사전 검증
메모리 부족	• 목표 모델·시퀀스 길이의 메모리 풋프린트 사전 산출 • 시스템 메모리 = GPU 메모리 총합의 2배 이상 • 옵티마이저 상태 분산(ZeRO/FSDP) 적용 가능성 검토
스토리지 I/O	• GPU당 지속 Read 대역폭 KPI 정의(예: 2GB/s/GPU) • GPUDirect Storage 지원 파일시스템 • 작은 파일은 샤딩·아카이브 포맷(WebDataset 등)으로 통합
열 스로틀링	• 랙당 열 부하(kW) 사전 산출 • Hot/Cold Aisle 격리 • 노드 배치 시 동시 풀로드 시나리오 시뮬레이션 • 랙 상·중·하 온도 센서 배치
체크포인트 오버헤드	• 비동기·분산 체크포인트(예: DCP) 적용 • 체크포인트 전용 스토리지 대역폭 확보 • 체크포인트 주기와 손실 비용의 RPO(recovery point objective) 기반 설계

05 확장성 및 운영

5.1 수평 확장(Scale-out) 전략

5.2 자원 활용 효율

05 확장성 및 운영

AI 인프라의 가치는 도입 시점이 아니라 운영 1년 차 이후의 총 소유 비용(Total Cost of Ownership, TCO)과 사용성에서 결정됩니다. 초기 구축이 완료된 후에는 워크로드 변화에 따른 확장 비용, 새 자원이 늘어났을 때 운영 부담의 증가, 그리고 도입된 GPU 자산이 실제로 얼마나 가동되는지가 인프라의 실효 가치를 좌우합니다. 본 장은 확장성과 운영 효율을 동시에 보장하는 NHN FactoryX의 운영 모델을 설명합니다.

5.1 수평 확장(Scale-out) 전략

클러스터의 확장은 GPU 수만 늘리는 단순한 작업이 아닙니다. 새 자원이 기존 워크로드의 성능과 운영 자동화 범위에 동일한 수준으로 편입되어야 하며, 증설 작업 자체가 기존 운영을 멈추거나 큰 변경을 요구해서는 안 됩니다. NHN FactoryX는 이를 위해 사전 정의된 단위로 증설하는 방식을 채택하고, 워크로드 특성에 따라 수평 확장과 수직 확장을 명확히 분리하여 적용합니다.

5.1.1 Scalable Unit 기반 증설

NHN FactoryX의 GPU 클러스터는 사전 정의된 Scalable Unit을 최소 증설 단위로 사용합니다. Scalable Unit은 다음을 포함합니다.

- GPU 노드 N대(표준 구성)
- Rail-Optimized Leaf 스위치 그룹
- 스토리지 클라이언트 라이선스·대역폭 할당
- 관리 평면 IP·DNS·인증서 자동 발급 자원

이러한 표준화는 두 가지 가치를 제공합니다. 첫째, 증설 시 예측 가능한 성능과 비용을 보장합니다. 둘째, 운영 자동화(IaC, 이미지 배포, 헬스 체크)가 새로운 단위에 동일하게 적용되어 운영 부담이 비례적으로 증가하지 않습니다.

5.1.2 수평 확장과 수직 확장의 분리

확장 전략은 워크로드가 요구하는 자원의 성격에 따라 두 방향으로 나뉩니다. ‘처리할 작업을 더 많이 감당하는 것’과 ‘하나의 작업을 더 크게 돌리는 것’은 늘려야 할 자원이 다르므로, 대응 방식도 구분됩니다.

- **수평 확장(Scale-out)**: 초거대 모델 학습이나 멀티테넌트 환경처럼 동시에 처리할 워크로드 수와 총 연산량이 증가하는 경우에 해당하며, Scalable Unit 단위 증설로 대응합니다.
- **수직 확장(Scale-up)**: 단일 모델의 메모리와 대역폭이 부족한 경우에 해당하며, NVLink 도메인을 확대(예: NVL72)하거나 차세대 GPU를 도입하여 노드 단위의 자원 상한을 끌어올리는 방식으로 대응합니다.

5.2 자원 활용 효율

자원 활용 효율은 GPU가 단순히 켜져 있는 시간이 아니라, 실제로 의미 있는 연산을 수행하는 시간의 비율로 측정됩니다. 같은 GPU 자산이라도 스케줄링·격리·우선순위 정책에 따라 가동률이 크게 달라지며, 운영 자동화의 수준이 높을수록 비정상 작업을 사전에 식별하거나 교정하여 효율 손실을 줄일 수 있습니다. NHN FactoryX는 다음과 같은 운영 메커니즘으로 자원 활용 효율을 끌어올립니다.

- **Gang Scheduling**: 분산 학습 작업이 요구하는 GPU 그룹 전체가 동시에 확보되어야만 실행되도록 보장하여, 부분 점유로 인한 자원 낭비를 차단합니다.
- **토폴로지 인지 스케줄링**: 스케줄러가 InfiniBand(인피니밴드) 토폴로지를 인식하고, 같은 작업의 GPU를 가능한 한 동일 Rail/Leaf 도메인 안에 배치해 통신 병목을 줄입니다.
- **Preemption/Priority Tier**: 긴급 작업이 낮은 우선순위 작업을 일시 중단할 수 있도록 다단계 큐를 운영하고, 중단된 작업은 자동 재개합니다.
- **MIG/Time-Slicing**: 추론과 개발 워크로드의 경우 단일 GPU를 분할해 다수 작업이 공유하도록 하여 GPU 가동률을 끌어올립니다.
- **관측 기반 자동 튜닝**: MFU(Model FLOPS Utilization)·step time·통신 시간 비율 등을 자동 수집하여, 비정상 작업을 사전에 식별하고 대응 가이드를 운영자에게 제공합니다.

본 장에서는 클러스터를 사전 정의된 단위로 확장하여 증설 시 성능을 일정하게 유지하고 운영 부담을 줄이는 방법과, 스케줄링·격리·관측 메커니즘으로 GPU 자산의 실제 가동률을 끌어올리는 방법을 살펴보았습니다. 이어지는 [6장](#)에서는 이와 같이 확장성과 운영 효율을 동시에 확보한 인프라가 실제로 산업 현장에서 어떻게 사용되는지를 사례를 통해 살펴봅니다.

NHN FactoryX

06 활용 사례

- 6.1 NHN FactoryX: 엑사스케일 클러스터
- 6.2 기업 프라이빗 AI 인프라 구축 사례

06 활용 사례

본 장에서는 앞에서 다룬 인프라 설계·운영 원칙이 실제 서비스에서 어떻게 구현되고 있는지를 두 가지 활용 사례로 살펴봅니다. 첫 번째는 다수 기업·기관이 동일 인프라를 공유하는 멀티테넌트 GPUaaS(GPU as a Service) 환경이며, 두 번째는 단일 기업이 전용 클러스터를 GPUaaS 형태로 활용하는 기업 프라이빗 AI 인프라 환경입니다. 두 사례는 자원 격리와 운영 자동화 요구사항이 다르지만, 동일한 GPUaaS 플랫폼 아키텍처 위에서 모두 구현된다는 점에서 NHN FactoryX 운영 모델의 확장성을 보여줍니다.

6.1 NHN FactoryX: 엑사스케일 클러스터

대규모 GPU 클러스터는 제공 방식에 따라 인프라 운영 구조가 달라지며, 이는 보안 통제와 서비스 안정성에 직접적인 영향을 미칩니다. 베어메탈 방식은 GPU 자원을 물리 서버 단위로 직접 할당하는 구조로, 사용자가 실행 환경 전반을 직접 구성할 수 있어 하드웨어 성능을 최대한 활용할 수 있습니다. 다만 물리 노드 수준의 관리자 권한이 사용자에게 직접 부여되는 구조상, 다수의 사용자가 동일 인프라를 공유할 경우 테넌트 간 보안 격리가 보장되지 않으며, 테넌트별 환경 구성이 상이하여 장애 발생 시 인프라 차원의 일관된 운영 정책 적용이 어렵다는 한계가 있습니다.

NHN FactoryX 서울 데이터 센터는 이러한 한계를 극복하기 위해 957개 GPU 노드로 이루어진 GPU 인프라 상위 계층에 GPUaaS 플랫폼을 구축하는 방식을 채택했습니다. 해당 플랫폼은 스케줄링 시스템 및 컨테이너 환경을 통합 관리합니다. 자원 및 보안 통제는 테넌트(Tenant), 워크스페이스(Workspace), 프로젝트(Project)로 이어지는 계층적 구조를 기반으로 인프라 자원 전반에 일관되게 적용됩니다. 이 계층 구조에서 운영자는 사용자를 테넌트 단위로 구분하고 각 테넌트 하위에 프로젝트와 워크스페이스를 생성합니다. 테넌트별로 사용 가능한 컴퓨팅 자원은 논리적으로 제한되며, 사용자는 자신이 속한 테넌트 내에서만 자원을 사용할 수 있습니다.



그림 6-1 멀티테넌트 GPUaaS 환경 구성 예

GPUaaS 방식은 서비스 계층이 추가되는 구조적 특성상 네트워크 병목으로 인한 학습 성능 저하가 발생할 수 있습니다. NHN Cloud는 이러한 우려를 최소화하기 위해 컨테이너가 호스트 수준의 네트워크 성능을 유지하도록 했습니다. 실제 성능 검증을 위해 GPUaaS 플랫폼 환경에서 255노드 규모로 HPL(High Performance Linpack) 벤치마크를 수행한 결과, 베어메탈과 동일한 수준의 성능을 달성함을 확인했습니다. 이 결과는 GPUaaS 플랫폼 방식에서도 고성능 컴퓨팅 환경을 충분히 구현할 수 있음을 보여줍니다.

2026년 6월 기준 NHN FactoryX 서울 데이터 센터의 클러스터 1과 2에서는 산업계·학계·연구계를 포함한 기업 및 기관이 510대와 255대의 GPU 노드로 구성된 GPUaaS 플랫폼을 활용하고 있습니다. 각 기관은 독립된 워크스페이스 환경에서 자체 데이터와 학습 워크로드를 운영하고 있습니다. 구체적 운영 사례를 살펴보면, A기관은 멀티모달 AI와 LLM Multi-Agent 연구 및 학습을 통해 의료 분야 활용 기술을 개발하고 있으며, B기관은 자체 LLM 모델 학습 및 서비스 제공을 위한 다양한 실험과 검증을 수행하고 있습니다. 이 과정에서 A기관의 워크스페이스에서 장애가 발생하더라도 B기관의 학습 및 서비스 환경에는 영향을 미치지 않습니다. 예를 들어, A기관에서 대규모 학습 작업으로 인해 자원 사용량이 급증하거나 특정 서비스에 장애가 발생하더라도, B기관은 독립적으로 분리된 자원과 네트워크 환경에서 안정적으로 서비스를 운영할 수 있습니다.

이처럼 NHN FactoryX 서울 데이터 센터는 조직 간 보안 격리와 자원 통제라는 요구사항을 단일 플랫폼 위에서 충족하며, 다수의 사용자가 동일한 GPU 인프라를 안정적으로 공유하는 운영 환경을 실현하고 있습니다. 앞으로도 각 기관과 기업이 워크로드 특성에 맞게 GPU 자원을 안정적으로 활용할 수 있도록 운영 환경을 지속적으로 유지 및 개선해 나갈 예정입니다.

6.2 기업 프라이빗 AI 인프라 구축 사례

전사 AI 전략을 본격 추진하는 기업이 자체적으로 GPU 인프라를 구축하려는 경우 데이터 센터 설계와 전력·냉각 인프라 확보부터 최신 GPU 수급, 운영 인력 확보, 네트워크 패브릭 튜닝까지 막대한 시간과 비용이 소요됩니다. 또한 부서 또는 사업별로 AI 워크로드의 특성과 규모가 상이하여 단일 인프라 위에서 자원 분배와 운영 격리를 동시에 달성하기 어렵다는 한계가 있습니다. 외부 클라우드의 일반 GPU 인스턴스를 사용할 경우 인프라 확보 부담은 줄어들지만 전용 클러스터 수준의 성능과 격리를 보장하기 어려우며, 기업 고유의 보안 및 운영 정책을 유연하게 적용하기 어렵다는 제약 또한 발생합니다.

NHN Cloud는 이러한 제약을 동시에 해소할 수 있는 기업 프라이빗 AI 인프라를 제공합니다. 기업이 직접 데이터 센터와 GPU·네트워크 자원을 확보하는 대신 표준화된 GPU 클러스터를 전용으로 할당받고 그 위에 GPUaaS 플랫폼을 적용함으로써, 자체 구축의 시간·비용 부담은 줄이면서 전용 클러스터 수준의 성능과 보안 격리를 동시에 확보할 수 있습니다. 이러한 모델로 실제 운영 중인 사례 가운데 하나가 클러스터 3입니다.

클러스터 3은 192대의 GPU 노드로 구성된 GPU 클러스터로, 게임 산업 E기업과 AI 산업 F기업이 자원을 독립적으로 활용하고 있습니다. E기업은 GPU 노드 40여 대를 AI 모델 개발 및 운영에 활용하고 있으며, F기업은 GPU 노드 140여 대를 활용하여 대규모 언어 모델(Large Language Model, LLM) 학습을 수행하고 있습니다.

두 기업이 동일 클러스터를 공유하면서도 독립적인 운영이 가능한 것은 GPUaaS 플랫폼의 테넌트 격리 구조에 기반합니다. 각 기업과 내부 부서는 독립된 테넌트 단위로 자원을 할당받고, 웹 기반 관리 콘솔을 통해 타 테넌트와의 간섭 없이 LLM, 멀티모달, 대화형 AI 등 부서별 요구사항에 맞는 모델을 독립적으로 개발 및 운영합니다. 각 기업의 인프라 관리자 역시 동일한 웹 기반 플랫폼을 통해 전체 노드의 자원 현황을 통합 모니터링하고, 부서별 자원 할당 및 접근 권한을 일원화하여 관리합니다.

이처럼 NHN FactoryX는 단일 기업의 전사 AI 전략을 뒷받침할 수 있는 전용 GPU 클러스터를 GPUaaS 형태로 구축·운영함으로써 자체 인프라 구축의 시간 및 비용 부담을 줄이고 부서 간 독립 운영과 통합 관리를 동시에 충족하는 인프라 모델을 제시합니다. 앞으로도 기업의 AI 워크로드 규모와 특성에 맞추어 인프라를 유연하게 확장하고 운영 효율을 지속적으로 개선해 나갈 예정입니다.

본 장에서 살펴본 두 사례는 자원 격리 단위와 운영 주체가 다름에도 동일한 GPUaaS 플랫폼 아키텍처 위에서 구현됩니다. 이는 NHN FactoryX의 운영 모델이 학계, 연구, 산업 전반의 다양한 AI 워크로드 요구사항을 단일 인프라 표준으로 수용할 수 있음을 보여줍니다.

NHN FactoryX

07 결론

07 결론

생성형 AI와 대규모 언어 모델의 등장으로 엔터프라이즈 연산 규모는 기존 범용 클라우드 인프라가 가정해 온 범위를 크게 초과했습니다. 통신 및 오버헤드, 데이터 굾주림, 체크포인트 쓰기 병목, 전력 밀도 급증이라는 구조적 병목은 단일 하드웨어 성능 개선만으로는 해결할 수 없으며, 물리 인프라부터 소프트웨어 관리 계층까지 모든 레이어가 유기적으로 통합된 AI 전용 인프라를 요구합니다. 본 백서는 AI 전용 인프라의 설계 철학과 운영 역량을 풀스택 관점에서 체계적으로 정리하고, NHN FactoryX가 이를 어떻게 구현하고 있는지 기술했습니다.

2장 시스템 아키텍처에서는 GPU 클러스터 구성 및 고속 인터커넥트 설계, 전력 및 냉각 설계, GPU 동적 할당 기술에 대해 알아보았습니다. NHN FactoryX는 Scalable Unit 기반의 4계층 클러스터 구조를 채택하고, Compute·Storage·Management 세 패브릭을 물리적으로 분리하여 트래픽 간 간섭을 원천 차단합니다. Fat-Tree 기반 Rail-Optimized 토폴로지로 집합 통신의 종단 지연을 최소화하며, 노드 내부에서는 NVSwitch 기반 Full-Mesh가 GPU 간 균일한 대역폭을 보장하고, 노드 간에서는 InfiniBand의 RDMA·Adaptive Routing·SHARP이 분산 학습의 통신 병목을 구조적으로 완화합니다. 전력 및 냉각 설계에서 NHN FactoryX는 D2C 수랭식 냉각을 채택하여 공기 매개 단계를 완전히 제거하였으며, 프리쿨링 연계 운전으로 PUE 1.15 이하를 달성할 수 있을 것으로 기대됩니다. GPU Live는 GPU 동적 할당 기술로 Kubernetes 표준 생태계 위에서 GPU 자원을 단일 풀로 추상화하고, 3계층 멀티테넌트·빈 패킹·쿼터·오버부킹·자동 회수를 통합 운영하여 정적 할당의 활용률 한계를 극복합니다.

3장 하드웨어 구성에서는 GPU 노드의 표준 구성과 각 구성 요소의 선택 근거를 다루었습니다. NHN FactoryX는 CPU·메모리를 GPU에 데이터를 굾김 없이 공급하는 균형 비율로 설계하고, NUMA 인지 배치와 PCIe Gen5 레인 확보를 표준화하고 있습니다. 워크로드 특성에 따라 학습용 고성능 GPU 라인과 추론·파인튜닝용 GPU 라인을 분리 운영하며, 스토리지는 NVMe 캐시·병렬 파일 시스템·오브젝트 스토리지의 3계층 구조로 I/O 특성에 따라 분리합니다. 네트워크는 네 개의 물리적 평면으로 구성하여 각 트래픽 유형이 서로 영향을 주지 않도록 설계됩니다. 이러한 설계 원칙 위에서 NHN FactoryX는 성능 저하 패턴의 사전 차단과 Scalable Unit 기반의 운영 자동화를 통해 클러스터의 실효 가동률을 확보합니다.

NHN FactoryX가 지원하는 각 레이어는 독립적으로도 의미가 있지만, NHN FactoryX의 핵심 가치는 이들이 하나의 통합 운영 모델로 결합된다는 점에 있습니다. 수랭식 데이터 센터, HPC 클러스터, GPUaaS 플랫폼, AI Agent as a Service가 단일 공간에서 제공되며, 동일한 GPUaaS 아키텍처 위에서 멀티테넌트 공유 환경과 기업 프라이빗 전용 환경이 모두 구현됩니다. **6장 활용 사례**에서 확인한 바와 같이 957개 GPU 노드로 구성된 GPU 인프라에서 GPUaaS 플랫폼이 베어메탈 동등 수준의 성능을 달성하면서도 조직 간 보안 격리와 자원 통제를 양립하고 있습니다. 기업 프라이빗 환경에서도 동일한 운영 모델이 적용되어 자체 구축 대비 시간과 비용 부담을 줄이면서 전용 클러스터 수준의 성능과 독립성을 확보하고 있습니다.

AI 인프라는 현재진행형입니다. GPU 세대 전환에 따른 전력 밀도 증가, Kubernetes DRA를 비롯한 자원 관리 표준의 진화, NPU 등 이종 가속기의 등장은 인프라 설계의 전제 조건을 지속적으로 변화시킵니다. NHN FactoryX는 차세대 GPU 플랫폼(B300 이후)까지의 상위 호환성을 전제로 수랭식 인프라를 설계하였으며, GPU Live의 동적 할당 모델을

이종 가속기로 확장하기 위한 기술 검토를 진행하고 있습니다. 보장 할당량 도입을 통한 SLA 정량 보장, 토폴로지 인식 배치 고도화, 학습 및 추론 동시 배치 정책 정교화 등 운영 모델의 기능 고도화 역시 단계적으로 추진되고 있습니다.

본 백서가 AI 인프라 도입을 검토하는 조직의 기술 의사결정자와 대규모 AI 시스템을 설계·운영하는 아키텍트 및 엔지니어에게 실질적인 기술 레퍼런스로 활용되기를 기대합니다.

08 부록

- 8.1 GPU 모델별 사양
- 8.2 용어 정의
- 8.3 참고 문헌

08 부록

8.1 GPU 모델별 사양

본 백서에서 다룬 GPU 모델별 사양을 확인할 수 있습니다.

표 8-1 GPU 모델별 사양 비교표⁶

항목	A100 SXM	H100 SXM	H200 SXM	B200 SXM	B300 SXM
아키텍처	Ampere	Hopper	Hopper	Blackwell	Blackwell Ultra
GPU당 HBM	80GB	80GB	141GB	180GB	270GB
HBM 대역폭	2TB/s	3.35TB/s	4.8TB/s	7.7TB/s	7.7TB/s
FP64 Tensor	19.5TFLOPS	67TFLOPS	67TFLOPS	37TFLOPS	1.25TFLOPS
TF32 Tensor	312TFLOPS	989TFLOPS	989TFLOPS	2.2PFLOPS	2.2PFLOPS
BF16/FP16 Tensor	624TFLOPS	1,979TFLOPS	1,979TFLOPS	4.5PFLOPS	4.5PFLOPS
FP8 Tensor	미지원	3,958TFLOPS	3,958TFLOPS	9PFLOPS	9PFLOPS
FP4 Tensor	미지원	미지원	미지원	18PFLOPS	18PFLOPS
NVLink 대역폭	600GB/s	900GB/s	900GB/s	1,800GB/s	1,800GB/s
GPU TDP	400W	700W	700W	1,000W	1,400W
주력 연산	FP16/BF16	FP64/FP16	FP64/FP16	FP8	FP4
대표 워크로드	범용 AI 학습	과학 계산, LLM 학습	과학 계산, 대형 모델 추론	LLM 학습·추론	LLM 추론 서비스

6 2026년 6월 기준 NVIDIA 공식 홈페이지에 공개된 자료를 기반으로 정리한 내용으로 공급사의 사정에 따라 세부 사양이 변경될 수 있습니다.

8.2 용어 정의

본 백서에서 사용된 기술 용어에 대한 설명을 참고할 수 있습니다.

표 8-2 용어 정의

용어	설명
멀티테넌시(Multitenancy)	여러 사용자가 클라우드에서 동일한 애플리케이션 및 컴퓨팅 리소스를 공유하는 소프트웨어 아키텍처. 하나의 물리 인프라(클러스터)를 여러 조직·사용자(테넌트)가 공유하면서도, 각 테넌트의 자원·데이터·접근 권한이 서로 격리되는 운영 모델
베어메탈(Bare Metal)	클라우드 환경에서 가상화되지 않은 물리 서버에 운영체제를 단독으로 설치하고 응용 프로그램을 직접 실행하는 개념. 동일한 물리적 서버 자원을 공유하여 사용하는 방식의 가상 서버를 보완하여 주로 데이터베이스와 같이 성능과 안정성에 민감한 서비스에서 활용
빈패킹(Bin Packing)	신규 워크로드를 이미 사용 중인 노드에 우선 배치하여 빈 노드를 보존하는 스케줄링 전략. 후속 대규모 작업이 연속된 GPU 자원을 확보할 수 있도록 단편화를 최소화함
서멀 스로틀링(Thermal Throttling)	GPU나 CPU의 온도가 안전 임계치를 초과하면 하드웨어 보호를 위해 자동으로 클럭 속도를 낮추는 메커니즘. 성능이 강제로 제한되므로 클러스터 전체 학습 속도에 영향을 미침
선점(Preemption)	우선순위가 높은 작업이 실행될 수 있도록, 현재 실행 중인 낮은 우선순위 작업을 강제로 종료하고 해당 자원을 재할당하는 메커니즘. 긴급 작업의 즉시 실행을 보장함
수직 확장(Scale-up)	단일 서버에 CPU, 메모리, 스토리지 등을 추가하여 서버 자체의 사양을 업그레이드하는 확장 방식
수평 확장(Scale-out)	시스템이 사용하는 서버 또는 노드의 개수를 늘려 시스템의 처리량을 높이는 방식
시분할(Time-Slicing)	하나의 물리 GPU를 여러 컨테이너가 시간 단위로 번갈아 사용하는 소프트웨어 기반 GPU 공유 방식. 별도 하드웨어 변경 없이 적용 가능하나, 메모리와 장애 영역이 격리되지 않음
오버부킹(Overbooking)	사용자에게 발급한 쿼터 합계를 물리 자원 총량보다 크게 설정하는 정책. 모든 사용자가 동시에 최대 한도까지 사용하지 않는 통계적 특성을 활용하여 클러스터 전체 활용률을 높임
인터커넥트(Interconnect)	서버 내부의 GPU 간 또는 서버 간 데이터를 전송하는 고속 연결 기술의 총칭
지연 시간(Latency)	입력 신호에 대해 출력 신호가 지연되어 응답하는 시간 간격 또는 패킷 전송을 요청한 시간과 전송이 시작될 때까지의 시간 간격. AI 클러스터에서 레이턴시가 높으면 GPU 간 동기화가 느려져 전체 학습 속도가 저하됨

용어	설명
체크포인팅(Checkpointing)	장시간 진행되는 AI 학습 도중, 모델의 현재 상태(가중치, 옵티마이저 상태 등)를 주기적으로 저장하는 작업. 하드웨어 장애 발생 시 마지막 체크포인트부터 학습을 재개할 수 있어 복구 시간을 단축함
컴퓨팅 패브릭(Compute Fabric)	GPU 노드 간 분산 연산 트래픽을 전달하는 InfiniBand/RoCE 기반의 고대역폭·저지연 RDMA 백엔드 네트워크
쿼터(Quota)	사용자·팀·프로젝트가 사용할 수 있는 자원(GPU, 메모리 등)의 상한. 쿼터를 초과하는 자원 요청은 차단되어, 특정 사용자가 전체 클러스터 자원을 독점하는 것을 방지
토폴로지(Topology)	네트워크에서 노드(장치)와 링크(통신 회선) 간의 물리적·논리적 연결 구조
Adaptive Routing(적응형 라우팅)	InfiniBand 스위치가 실시간 네트워크 혼잡 정보를 기반으로 데이터 전송 경로를 동적으로 변경하는 기능. 모든 GPU가 동시에 통신하는 집합 통신 패턴에서 특정 경로에 부하가 집중되는 것을 완화함
All-Reduce	분산 학습에서 여러 GPU가 각각 계산한 그레이디언트를 모두 합산(Reduce)한 뒤, 그 결과를 다시 모든 GPU에 배포(All)하는 집합 통신 연산. 모든 GPU가 동일한 학습 결과를 공유하기 위해 반드시 필요한 과정
ASHRAE	미국 냉난방공조학회(American Society of Heating, Refrigerating and Air-Conditioning Engineers). 데이터 센터 냉각 설계 표준(TC 9.9)을 제정하며, 냉각 등급(A1~A4, H1, W1~W5 등)의 운전 온도 범위를 규정
CDU(Cooling Distribution Unit, 냉각 분배 장치)	건물 측 1차 냉각수 루프와 IT 장비 측 2차 루프를 물리적으로 분리하고, IT 부하에 적합한 온도·압력·수질 조건으로 냉각수를 공급하는 중간 열교환 장치
D2C(Direct-to-Chip, 직접 칩 냉각)	CPU나 GPU 같은 발열 칩에 쿨드 플레이트를 직접 밀착시켜 냉각수로 열을 제거하는 수랭식 냉각 방식. 공기를 매개로 하지 않아 열 저항이 낮고, 랙당 120kW 이상의 고밀도 발열을 처리할 수 있다.
DRA(Dynamic Resource Allocation, 동적 자원 할당)	Kubernetes 1.34에서 공식적으로 도입된 차세대 디바이스 자원 관리 표준. 기존 Device Plugin 모델을 대체하여 GPU, NPU 등 가속기 자원의 동적 청구·할당·해제를 표준화
Fat-Tree	트리 구조의 네트워크에서 상위 계층으로 갈수록 링크 대역폭을 넓게(Fat) 확보하여, 어떤 두 노드 사이에서도 동일한 유효 대역폭을 보장하는 설계. AI 클러스터에서 모든 GPU의 동시 통신을 지원하기 위해 널리 사용됨
Gang Scheduling(갱 스케줄링)	분산 학습 작업의 모든 워커(Worker)가 동시에 자원을 확보했을 때만 일괄 실행하는 스케줄링 정책. 일부 워커만 실행되어 나머지를 기다리며 자원을 낭비하는 상황을 구조적으로 방지함
GPU(Graphics Processing Unit, 그래픽 처리 장치)	대규모 병렬 연산에 특화된 프로세서. 원래 그래픽 렌더링용으로 설계되었으나, 수천 개의 코어를 활용한 행렬 연산 능력 덕분에 AI 모델의 학습과 추론에 핵심적으로 사용됨

용어	설명
GPUaaS(GPU as a Service)	GPU 자원을 물리 서버 단위로 직접 제공하는 대신, 플랫폼 계층에서 가상화·격리하여 서비스 형태로 제공하는 방식. 다수의 사용자가 동일 클러스터를 안전하게 공유하면서도 독립적인 운영 환경을 보장받을 수 있음
GPUDirect Storage(GDS)	스토리지에서 GPU 메모리로 데이터를 CPU를 거치지 않고 직접 전송하는 NVIDIA 기술
HBM(High Bandwidth Memory, 고대역폭 메모리)	GPU 칩 위에 수직으로 적층하여 초고속 데이터 전송을 실현하는 메모리 기술. 기존 GDDR 대비 수 배 높은 대역폭을 제공하며, AI 학습에서 대규모 모델 파라미터를 빠르게 처리하는 데 필수적
HPL(High Performance Linpack)	대규모 선형 방정식을 풀어 슈퍼컴퓨터의 부동소수점 연산 성능을 측정하는 벤치마크. TOP500 슈퍼컴퓨터 순위의 공식 측정 기준으로 사용되며, GPU 클러스터의 실제 연산 효율을 검증하는 데 활용
IaC(Infrastructure as Code, 코드형 인프라)	서버, 네트워크, 스토리지 등 인프라 구성을 수동 작업 대신 코드(스크립트)로 정의하고 자동으로 배포·관리하는 방식. 증설 시 동일한 환경을 반복적으로 재현할 수 있어 운영 일관성을 보장함
InfiniBand	노드 간 RDMA 기반 고성능 네트워크. AI 클러스터의 표준 East-West 패브릭
Kubernetes	컨테이너화된 애플리케이션을 배포, 관리 및 크기 조정하기 위한 오픈소스 오케스트레이션 플랫폼. Google이 2014년 오픈소스화했으며 K와 S 사이에 있는 8자를 줄여 K8s라고도 표기
LLM(Large Language Model, 대규모 언어 모델)	수천억 개 이상의 파라미터를 가진 인공지능 언어 모델. 대량의 텍스트 데이터를 학습하여 자연어 이해, 생성, 번역, 요약 등 다양한 언어 작업을 수행
MFU(Model FLOPS Utilization, 모델 FLOPS 활용률)	AI 학습 시 GPU가 이론적 최대 연산 성능 대비 실제로 달성하는 유효 연산 비율
MIG(Multi-Instance GPU)	단일 GPU를 격리된 다중 인스턴스로 분할하는 NVIDIA 기능
MPS(Multi-Process Service)	NVIDIA CUDA 런타임이 제공하는 GPU 공유 메커니즘. 여러 프로세스의 CUDA 작업을 하나의 GPU 위에서 동시에 실행하여 컨텍스트 전환 비용을 줄이고 처리량을 향상시킴
NUMA(Non-Uniform Memory Access, 비균일 메모리 접근)	멀티소켓 서버에서 각 CPU 소켓이 자신에게 가까운(로컬) 메모리에는 빠르게, 먼(리모트) 메모리에는 느리게 접근하는 메모리 구조. GPU와 NIC를 같은 NUMA 도메인에 배치하면 데이터 전송 지연이 줄어듦
NVLink	노드 내 GPU 간 고대역폭·저지연 인터커넥트(NVIDIA 전용)
NVMe(Non-Volatile Memory Express)	SSD와 같은 비휘발성 저장장치를 위해 설계된 고성능 인터페이스 프로토콜. 기존 SATA/SAS 대비 훨씬 낮은 지연과 높은 처리량을 제공하여, AI 학습 데이터의 빠른 로딩에 활용
NVSwitch	NVLink를 스위칭해 모든 GPU가 full-bandwidth로 통신하게 하는 스위치

용어	설명
OOM(Out of Memory, 메모리 부족)	GPU 또는 시스템 메모리가 할당 요청을 수용할 수 없을 때 발생하는 오류. AI 모델의 파라미터, 옵티마이저 상태, 활성화 데이터의 합이 가용 메모리를 초과하면 학습이 중단됨
Oversubscription(오버서브스크립션)	네트워크에서 하위 스위치(Leaf)의 총 대역폭 합이 상위 스위치(Spine)로의 업링크 대역폭보다 큰 상태. 1:1이면 비차단(Non-blocking) 구성이며, 비율이 커질수록 동시 통신 시 혼잡 위험이 높아짐
PCIe(Peripheral Component Interconnect Express)	컴퓨터 내부에서 CPU, GPU, NIC, SSD 등 주요 부품 간 데이터를 전송하는 범용 고속 인터페이스 규격. 최신 Gen5는 레인당 32GT/s, x16 구성 시 약 64GB/s의 단방향 대역폭을 제공
PUE(Power Usage Effectiveness, 전력 사용 효율)	데이터 센터의 총 전력 소비량을 IT 장비가 사용하는 전력으로 나눈 값. 1.0에 가까울수록 냉각·조명 등 부대 전력이 적어 에너지 효율이 높다. AI 전용 데이터 센터는 PUE 1.15 이하를 목표로 함
Rail-Optimized	노드의 N번째 GPU가 동일 N번째 Leaf 스위치 그룹에 연결되도록 정렬한 토폴로지
RBAC(Role-Based Access Control, 역할 기반 접근 제어)	역할에 의해 자원에 대한 접근 권한이 결정되는 접근 제어 방식. 조직의 업무 기능 및 목적에 따라 역할을 생성하고, 역할에 부여된 접근 권한에 따라 역할 수행에 필요한 최소한의 자원에 접근할 수 있도록 함
RDMA(Remote Direct Memory Access, 원격 직접 메모리 접근)	CPU를 거치지 않고 한 서버의 메모리에서 다른 서버의 메모리로 데이터를 직접 전송하는 기술. CPU 부하를 줄이고 마이크로초 단위의 초저지연 통신을 가능하게 함
RoCE(RDMA over Converged Ethernet)	기존 이더넷 네트워크 위에서 RDMA 기능을 구현하는 프로토콜. InfiniBand 전용 네트워크 없이도 낮은 지연 시간의 통신이 가능하며, 기존 이더넷 인프라를 활용할 수 있다는 장점이 있음
Scalable Unit	동일한 토폴로지·구성·운영 자동화가 적용되는 클러스터 최소 증설 단위
SHARP(Scalable Hierarchical Aggregation and Reduction Protocol)	All-Reduce 같은 집합 연산의 일부를 네트워크 스위치 내부에서 직접 수행하는 NVIDIA의 인네트워크 컴퓨팅 기술. 데이터가 서버까지 왕복할 필요 없이 스위치 단에서 합산되므로 통신 시간이 대폭 단축됨
TCO(Total Cost of Ownership, 총 소유 비용)	인프라의 구매 비용뿐 아니라 설치, 운영, 유지보수, 전력, 냉각, 인력 등 전체 수명 주기에 걸쳐 발생하는 모든 비용의 합계. 인프라 도입 의사결정의 핵심 지표
TFLOPS(Tera Floating Point Operations Per Second)	GPU 연산 성능의 기본 측정 단위로, 1초에 수행할 수 있는 부동소수점 연산 횟수를 조 단위로 나타낸 것
vGPU(Virtual GPU, 가상 GPU)	NVIDIA가 가상화 환경을 위해 제공하는 GPU 분할 기술. 하이퍼바이저 계층에서 물리 GPU를 여러 가상 GPU로 나누어 개별 가상 머신에 할당한다. 별도 상용 라이선스가 필요

8.3 참고 문헌

01 개요

- Dell, “Understanding LLM GPUs Clusters Fabrics Traffic For Networkers”, Dell Blog, 2025. 9. 16., <https://www.dell.com/en-us/blog/understanding-llm-gpus-clusters-fabrics-traffic-for-networkers/>
- NVIDIA, “Fast AI Data Preprocessing with NVIDIA DALI”, NVIDIA Developer Blog, 2019. 1. 28., <https://developer.nvidia.com/blog/fast-ai-data-preprocessing-with-nvidia-dali/>
- HPE Juniper, “Thermal Management in AI Data Centers: Challenges and Solutions”, 2024. 12. 9., <https://blogs.juniper.net/en-us/ai-data-center-networking/thermal-management-in-ai-data-centers-challenges-and-solutions>
- NVIDIA, “Chill Factor: NVIDIA Blackwell Platform Boosts Water Efficiency by Over 300x”, NVIDIA Blog, 2025. 4. 22., <https://blogs.nvidia.com/blog/blackwell-platform-water-efficiency-liquid-cooling-data-centers-ai-factories/>

02 시스템 아키텍처

- NVIDIA, “Network Fabrics”, DGX SuperPOD Reference Architecture Featuring NVIDIA DGX GB200, <https://docs.nvidia.com/dgx-superpod/reference-architecture-scalable-infrastructure-gb200/latest/network-fabrics.html>
- NVIDIA, “Overview of NCCL”, NCCL Documentation, <https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/overview.html>
- C. E. Leiserson, “Fat-Trees: Universal Networks for Hardware-Efficient Supercomputing”, IEEE Transactions on Computers, vol. 34, no. 10, pp. 892–901, Oct. 1985. doi:10.1109/TC.1985.6312192
- NVIDIA, “NVIDIA NVLink and NVIDIA NVSwitch Supercharge Large Language Model Inference”, NVIDIA Technical Blog, <https://developer.nvidia.com/blog/nvidia-nvlink-and-nvidia-nvswitch-supercharge-large-language-model-inference/>
- ASHRAE TC 9.9, Thermal Guidelines for Data Processing Environments, 5th Edition (2021); ASHRAE TC 9.9, Datacom Encyclopedia (2024).
- A. Almoli, A. Thompson, N. Kapur, J. Summers, H. Thompson, and G. Hannah, “Computational Fluid Dynamic Investigation of Liquid Rack Cooling in Data Centres”, Applied Energy, vol. 89, no. 1, pp. 150–155, 2012.
- Q. Weng et al., “MLaaS in the Wild: Workload Analysis and Scheduling in Large-Scale Heterogeneous GPU Clusters”, Proc. USENIX NSDI, 2022.
- NVIDIA, “k8s-device-plugin: Shared Access to GPUs”, GitHub, <https://github.com/NVIDIA/k8s-device-plugin>
- NVIDIA, “NVIDIA Virtual GPU (vGPU) Software”, NVIDIA Documentation, <https://docs.nvidia.com/vgpu/>
- NVIDIA, “NVLink & NVLink Switch”, <https://www.nvidia.com/en-us/data-center/nvlink/>
- NVIDIA, “NCCL Documentation”, <https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/>

- Kubernetes SIG-Scheduling, “Coscheduling Plugin”, [kubernetes-sigs/scheduler-plugins](https://github.com/kubernetes-sigs/scheduler-plugins), <https://github.com/kubernetes-sigs/scheduler-plugins>
- E. G. Coffman, M. R. Garey, and D. S. Johnson, “Approximation Algorithms for Bin Packing: A Survey”, in *Approximation Algorithms for NP-Hard Problems*, 1996.
- A. Verma et al., “Large-Scale Cluster Management at Google with Borg”, *Proc. EuroSys*, 2015; V. K. Vavilapalli et al., “Apache Hadoop YARN: Yet Another Resource Negotiator”, *Proc. SOCC*, 2013.
- A. Verma et al., “Large-Scale Cluster Management at Google with Borg”, *Proc. EuroSys*, 2015; Kubernetes, “Pod Priority and Preemption”, *Kubernetes Documentation*, <https://kubernetes.io/docs/concepts/scheduling-eviction/pod-priority-preemption/>
- Kubernetes, “Dynamic Resource Allocation”, *Kubernetes Documentation*, <https://kubernetes.io/docs/concepts/scheduling-eviction/dynamic-resource-allocation/>; KEP-4381

08 부록

- NVIDIA, “Blackwell Architecture”, <https://www.nvidia.com/en-us/data-center/technologies/blackwell-architecture/>
- NVIDIA, “Time-Slicing GPUs in Kubernetes”, *GPU Operator Documentation*, <https://docs.nvidia.com/datacenter/cloud-native/gpu-operator/latest/gpu-sharing.html>
- NVIDIA, “End-to-End High-Speed Ethernet and InfiniBand Solutions”, <https://www.nvidia.com/en-us/networking/products/>
- NVIDIA, “GPUDirect Storage: A Direct Path Between Storage and GPU Memory”, *NVIDIA Developer Blog*, 2019. 8. 6., <https://developer.nvidia.com/blog/gpudirect-storage/>
- NVIDIA, “Accelerated InfiniBand Solutions for HPC”, <https://www.nvidia.com/en-us/networking/products/infiniband/>
- NVIDIA, “Data Centers Built for Advanced AI Reasoning”, <https://www.nvidia.com/en-us/data-center/>
- NVIDIA, “What are Large Language Models?”, *NVIDIA Glossary*, <https://www.nvidia.com/en-us/glossary/large-language-models/>
- NVIDIA, “Multi-Process Service”, *NVIDIA Documentation*, <https://docs.nvidia.com/deploy/mps/latest/index.html>
- NVIDIA, “GPUDirect RDMA”, *NVIDIA Documentation*, <https://docs.nvidia.com/cuda/gpudirect-rdma/index.html>
- NVIDIA, “Advancing Performance with NVIDIA SHARP In-Network Computing”, *NVIDIA Developer Blog*, 2024. 10. 25., <https://developer.nvidia.com/blog/advancing-performance-with-nvidia-sharp-in-network-computing/>
- NVIDIA, “Virtual GPU Solutions for AI and Graphics”, <https://www.nvidia.com/en-us/data-center/virtual-solutions/>
- Kubernetes, “Multi-tenancy”, *Kubernetes Documentation*, <https://kubernetes.io/docs/concepts/security/multi-tenancy/>
- Kubernetes, “Resource Bin Packing”, *Kubernetes Documentation*, <https://kubernetes.io/docs/concepts/>

scheduling-eviction/resource-bin-packing/

- Kubernetes, “Pod Priority and Preemption”, Kubernetes Documentation, <https://kubernetes.io/docs/concepts/scheduling-eviction/pod-priority-preemption/>
- Kubernetes, “Resource Quotas”, Kubernetes Documentation, <https://kubernetes.io/docs/concepts/policy/resource-quotas/>
- Kubernetes, “Dynamic Resource Allocation”, Kubernetes Documentation, <https://kubernetes.io/docs/concepts/scheduling-eviction/dynamic-resource-allocation/>
- Kubernetes, “Gang Scheduling”, Kubernetes Documentation, <https://kubernetes.io/docs/concepts/scheduling-eviction/gang-scheduling/>
- Kubernetes, “Overview”, Kubernetes Documentation, <https://kubernetes.io/docs/concepts/overview/>
- ASHRAE, <https://www.ashrae.org/>
- ASHRAE TC 9.9, “Datacom Encyclopedia”, <https://www.ashrae.org/technical-resources/bookstore/datacom-series>
- TTA 정보통신용어사전, “네트워크 토폴로지”, https://terms.tta.or.kr/dictionary/dictionaryView.do?word_seq=170524-3
- TTA 정보통신용어사전, “GPU”, <https://terms.tta.or.kr/dictionary/dictionaryView.do?subject=GPU>
- TTA 정보통신용어사전, “PUE”, https://terms.tta.or.kr/dictionary/dictionaryView.do?word_seq=061074-2
- TTA 정보통신용어사전, “TCO”, <https://terms.tta.or.kr/dictionary/dictionaryView.do?subject=TCO>
- B. Urgaonkar, P. Shenoy, and T. Roscoe, “Resource Overbooking and Application Profiling in Shared Hosting Platforms”, ACM SIGOPS Operating Systems Review, vol. 36, no. SI, pp. 239–254, Dec. 2002. <https://doi.org/10.1145/844128.844151>
- TOP500, “The Linpack Benchmark”, <https://www.top500.org/project/linpack/>
- NVM Express, <https://nvmexpress.org/>
- PCI-SIG, <https://pcisig.com/>
- InfiniBand Trade Association, “RoCE Initiative — Overview”, 2023. 2. 7., <https://www.roceinitiative.org/about-overview/>
- Wikipedia, “Fat tree”, https://en.wikipedia.org/wiki/Fat_tree
- NHN Cloud 용어 사전, <https://www.nhncloud.com/kr/resource/glossary>

NHN FactoryX

엔에이치엔클라우드(주)

13487 경기도 성남시 분당구 대왕판교로645번길 16, NHN 플레이뮤지엄

고객문의: 1588-7967 이메일: support@nhncloud.com

© NHN Cloud Corp. All rights reserved.